

A4.3

Machine learning approaches (HL)

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A4.3.1 Explain how linear regression is used to predict continuous outcomes
- ▶ A4.3.2 Explain how classification techniques in supervised learning are used to predict discrete categorical outcomes
- ▶ A4.3.3 Explain the role of hyperparameter tuning when evaluating supervised learning algorithms
- ▶ A4.3.4 Describe how clustering techniques in unsupervised learning are used to group data based on similarities in features
- ▶ A4.3.5 Describe how association rule learning techniques are used to uncover relations between different attributes in large data sets
- ▶ A4.3.6 Describe how an agent learns to make decisions by interacting with its environment in reinforcement learning
- ▶ A4.3.7 Describe the application of genetic algorithms in various real-world examples
- ▶ A4.3.8 Outline the structure and function of artificial neural networks (ANNs) and how multi-layer networks are used to model complex patterns in data sets
- ▶ A4.3.9 Describe how convolutional neural networks (CNNs) are designed to adaptively learn spatial hierarchies of features in images
- ▶ A4.3.10 Explain the importance of model selection and comparison in machine learning

Key information

Before proceeding into this chapter, it is important to note that the syllabus content statements above are limited to “Explain”, “Describe” and “Outline”.

This chapter intentionally contains additional detail that is beyond the syllabus, such as programming code samples demonstrating the use of various algorithms. **You do not need to be able to read or write programming code for the different algorithms presented here in your IB examinations.**

The additional detail has been provided due to machine learning being an extremely popular subset of Computer Science. In discussions with students, many expressed a desire to learn beyond descriptive theory for these topics, and a wish to know how these algorithms work and how to use them. Students also commonly express an intention to experiment with machine learning algorithms within their internal assessments. For these reasons, this introduction is more in-depth than is required solely for the examinations.

◆ Linear regression:

a machine learning algorithm that seeks a linear line of best fit for a given data set, from which extrapolations can be made.

A4.3.1 Supervised learning: linear regression

Linear regression refers to calculating the correlation and line (or plane) of best fit among the values of a data set, and then using the resulting *equation of the line* to make predictions for new, unseen data.

Linear regression is one of the earliest machine learning algorithms to be developed, and can even be calculated manually for limited data sets as they are purely mathematical constructs.

Top tip!

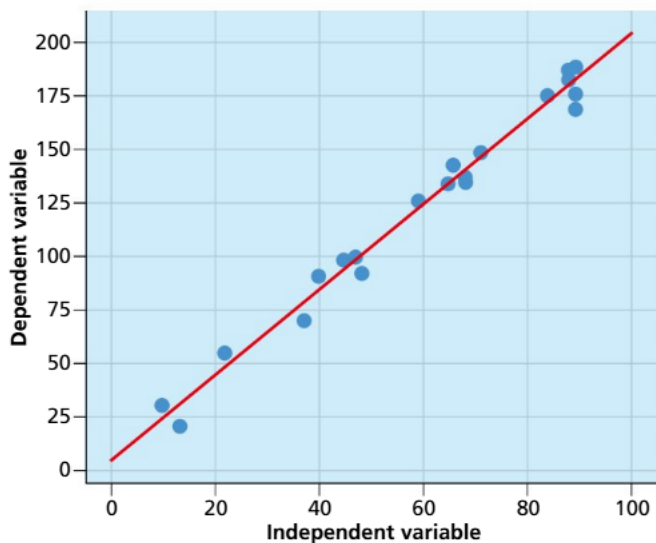
Always check the relationship between variables with scatter plots to see whether they are roughly linear. There is no point using linear regression if it doesn't fit the data.

Linear regression can be used to help answer such questions as:

- Given a person's height, is it possible to predict their weight?
- Given the number of hours a student studies for a test, is it possible to predict their result?
- Given the dollars spent on advertising a product, is it possible to predict the sales volume?
- Given the size of a home in square meters, is it possible to predict its sale price?

As hopefully you can infer from the name, linear regression is only suitable where there is a linear relationship between the independent and dependent variables.

The graph below illustrates a simple form of linear regression with one independent variable (the predictor) and one dependent variable (the response).



■ Linear regression example

To explore the process involved for linear regression of one independent and one dependent variable, the equation for the line of best fit can be calculated using the *least squares regression* line, which will minimize the distance between the line and individual data points.

By assigning the independent variable to x , and the dependent to y , you can use the standard equation of a line to make predictions.

$$y = a + bx$$

The slope, or gradient, of the line, b , represents the amount that the prediction will change for every increment of one in the independent variable. To calculate b , you find the sum of the difference between each point of the line from the mean:

$$b = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

The point of intercept, a , is the baseline value for the dependent variable when the independent variable is 0. To calculate a , take the coefficient multiplied by the mean of x and subtract it from the mean of y .

$$a = \bar{y} - b\bar{x}$$

As linear regression is such a common and popular task, both NumPy and scikit-learn libraries have tools built in to perform these calculations for you. The following example uses scikit-learn, as we will use the library a lot for other algorithms coming up.

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# Sample data
x = np.array([45, 48, 65, 68, 68, 10, 84, 22, 37, 88, 71, 89, 89, 13, 59, 66, 40, 88, 47, 89])
y = np.array([98, 92, 134, 135, 136, 30, 175, 54, 70, 182, 148, 169, 187, 20, 126, 142, 90, 186, 99, 176])
# Using scikit, calculate the model
# Convert the 1D array of 20 columns, to a 2D array of 20 rows 1 column each
x = x.reshape(-1, 1)
model = LinearRegression()
model.fit(x, y)
```

```

intercept = model.intercept_
slope = model.coef_[0]
# Using scikit, generate a prediction where the independent variable is 70
x_test = np.array([[70]]) # New single data point for prediction
y_test_predict = model.predict(x_test)
print(f"Prediction for independent variable value 70: {y_test_predict[0]}")
# Using matplotlib, plot the data and the line of best fit
x_line = np.array([[0], [100]])
y_line = np.array([intercept, [intercept+100*slope]])
plt.scatter(x, y, color="blue")
plt.plot(x_line, y_line, color="red", linewidth=2)
plt.xlabel("Independent variable")
plt.ylabel("Dependent variable")
plt.title("Linear Regression example")
plt.show()

```

See also

Reference the scikit-learn documentation for supervised linear regression by searching online for scikit-learn's "LinearRegression" (https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html).

◆ **R-squared value (or coefficient of determination):** a statistical measure that indicates how well the linear regression model fits the data points given.

Measuring accuracy

Assessing the accuracy of the model can be performed by calculating the **R-squared value**, also known as the **coefficient of determination**. It is a measure of the proportion of variation in values from the independent variable data, and what the model would have predicted for that point. A value close to 0 indicates the model does a poor job of explaining the relationship of the data, whereas a value close to 1 indicates the model does an excellent job of mapping the relationship of the data.

Why use a measure that relies on squaring? Why not just take the average of the absolute value of the difference between the predicted and actual values? There are a few reasons:

- Squaring means larger errors will become more pronounced than smaller errors. Absolute differences would treat all variations the same.
- Squaring results in a function that can be differentiated; a very handy benefit for optimization algorithms that rely on derivatives. Absolute values in a function cannot be differentiated.
- Squared differences are used within classical statistics with respect to assumptions around normalization, so it is convenient to stick with that approach.

There are approaches, other than R-squared value, that can be used, including:

- **adjusted R-squared:** modifies the formulae to account for the number of predictors in the model
- **mean squared error:** the average of the squares of the errors
- **mean absolute error:** the average of the absolute value of the errors
- **mean absolute percentage error:** the average of the absolute percentage errors of predictions.

With NumPy, the r-squared value needs to be calculated through each step, whereas scikit-learn has a built-in method for the task.

Python

```
# Method 1: Using numpy
y_predicted = model.predict(x)
y_residuals = y - y_predicted # Difference between actual and predicted values
ss_res = np.sum(y_residuals**2) # Sum of squares of the residuals
ss_tot = np.sum((y - np.mean(y))**2) # Total sum of squares
r_squared = 1 - (ss_res / ss_tot) # R-squared value
print(f"R-squared: {r_squared:.2f}")

# Method 2: Using scikit-learn
r_squared = model.score(x, y)
print(f"R-squared: {r_squared:.2f}")
```

■ Multidimensionality

The previous example illustrates simple linear regression where there is only one independent variable. In real-world applications, it is highly likely that you will want to model against several independent variables. This is known as “multiple linear regression”.

While most humans can mentally visualize data in two, or perhaps three, dimensions, it becomes extremely challenging to visualize beyond that. The good news is our software tools have no problem modelling the relationship across multiple dimensions. Here is an example using scikit for a linear regression model that has four independent variables.

Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Load the data and assign features (independent variables) to x, and the target
# (dependent variable) to y.
df = pd.read_csv("Multidimensional_example.csv")
x = df[["Feature_1", "Feature_2", "Feature_3", "Feature_4"]]
y = df["Target"]
# Train the linear regression model
model = LinearRegression()
model.fit(x, y)
# Generate prediction for new data point
new_point = np.array([[50, -150, 30, 100]])
prediction = model.predict(new_point)
print(f"Predicted target for the new sample: {prediction[0]}")
```

The Multidimensional_example.csv file can be downloaded from <https://github.com/paulbaumgarten/hodder-ibdp-computerscience>

The prediction for the new data point used in the code should be 510.7.

Top tip!

Regression with non-linear data?

If you have a data set you would like to fit to a non-linear function, the `scipy curve_fit()` function is what you are looking for.

For example, assuming you have NumPy arrays of `x_data` and `y_data` that you wish to fit to a quadratic function:

Python

```
import numpy as np
from scipy.optimize import curve_fit
x_data = [.....your data here.....]
y_data = [.....your data here.....]
def my_quadratic(x, a, b, c):
    return a * x**2 + b * x + c
params, params_covariance = curve_fit(my_quadratic,
x_data, y_data)
a, b, c = params # Extract the fitted coefficients
```

For more information, search online for `scipy.optimize.curve_fit` (https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.curve_fit.html).

A4.3.2 Supervised learning: classification techniques

◆ **Classification techniques:** where a machine learning model has been trained to identify, from a predefined list of categories, which category (or class) the input data would most likely be associated with.

One task frequently required of machine learning algorithms is to classify data as belonging to one of a given range of categories. Whereas the output prediction from linear regression is numeric, **classification** algorithms generally produce a non-numeric value to represent a category.

Categorization through machine learning is useful as it facilitates automation of decision-making processes and can be applied to a wide range of practical problems. Examples of everyday classification problems include:

- email spam detection (spam or not spam)
- medical diagnosis (disease or no disease)
- credit score (good credit risk or poor credit risk)
- image recognition (identifying what category of object is in the image)
- natural language processing / sentiment analysis (positive or negative sentiment)
- recommendation engine (what genre of movie to suggest next).

Two popular methods that do not require neural networks are k-nearest neighbours and decision trees.

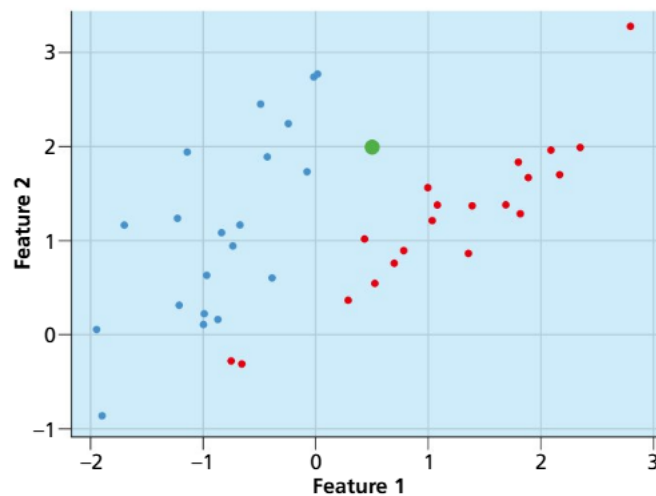
◆ **K-nearest neighbours:** where data points are categorized based on the categories of the nearest points around them in the data set; k is a variable representing how many of those nearest points should be used to “vote” and determine what category to assign the new value.

■ K-nearest neighbours

K-nearest neighbours is a machine learning technique that allows classification of data based on patterns learned from existing labelled data.

● Top tips!

- Normalize or standardize data because KNN is sensitive to the magnitude of data points.
- Choose an odd number for k when the number of classes is even to avoid tie situations.
- Experiment with different distance metrics (e.g. Euclidean, Manhattan) to see which performs best for your data set.



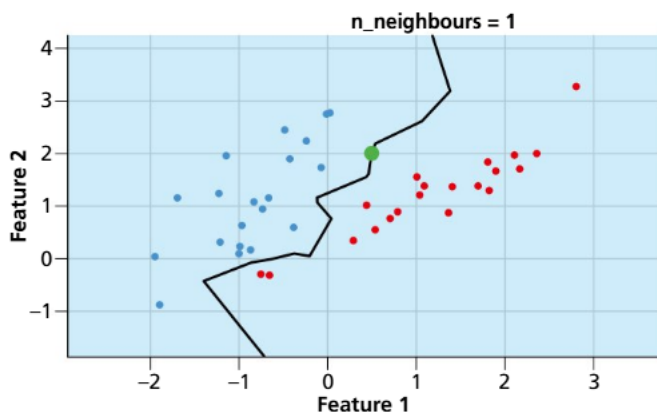
■ K-nearest neighbours

Consider the chart above, in which the data have been classified into either blue dots or red dots. The two axes represent the different features (variables) that have been measured. These can represent anything (such as size, weight, length, time, cost, review rating), provided they can be measured and plotted on a numeric scale.

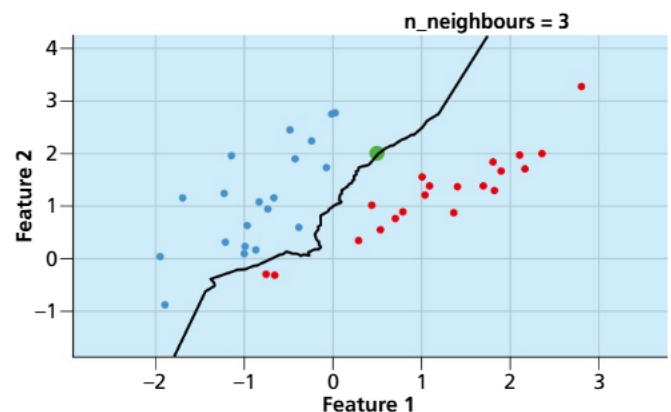
The green dot represents a new value that the model has not seen before. How can k-nearest neighbours be used to determine whether the green dot should be classed as belonging to the blue group or the red group?

With KNN, a value k is selected to represent how many nearby data points should be used to determine the prediction output. The algorithm will then determine the k -nearest points, and allow each of them to “vote” as to which final category the prediction should award.

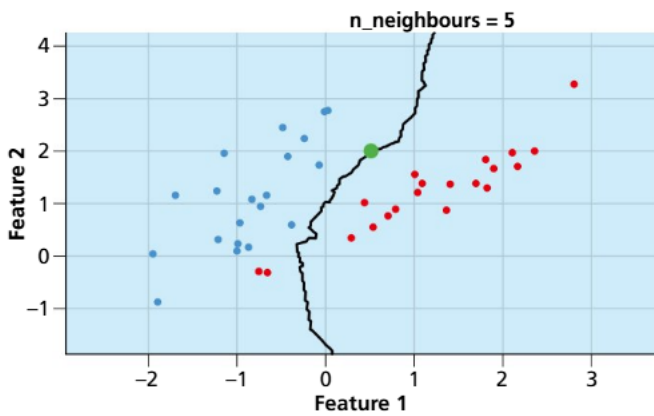
The following charts illustrate the decision boundary for different values of k .



■ KNN where $k = 1$



■ KNN where $k = 3$



In the examples here, values of k of 1, 3 and 5 have been used.

When $k = 1$, the line represents the boundary to the nearest single point of either category. Normally, this is susceptible to being influenced by outliers, so a value of $k = 3$ or $k = 5$ is more typical.

It is important when selecting between two categories to ensure that k is an odd number to avoid the situation where there could be a tie!

The data set for the charts shown here is the `knn_dataset.csv` file that can be downloaded from <https://github.com/paulbaumgarten/hodder-ibdp-computerscience>

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier

# Load the data
df = pd.read_csv("knn_dataset.csv")
x = df[["Feature_1", "Feature_2"]].values
y = df["Label"].values

# New, unknown point to classify
z = np.array([[0.4, 1.6]])

# Create and fit the KNN classifier with 3 neighbours
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(x, y)

# Plot the training points
plt.scatter(x[:, 0], x[:, 1], c=y)
plt.scatter(z[:, 0], z[:, 1], color="red", zorder=5)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("n_neighbors=3")
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.show()

# Make the prediction
predicted_category = knn.predict(z)
print("The predicted category for point z is:",
      predicted_category[0])
```

One example application of KNN is in the development of collaborative filtering recommendation systems. Consider a scatter plot for two movies that a wide number of users have reviewed and rated. The x axis may represent the ratings by each user for movie A, and the y axis may represent the ratings by each user for movie B.

Mark a new point on the plot for the target user, who has viewed movie A but not movie B. By referring to the other points on the scatter, KNN can infer what the target user would rate movie B. If the prediction is for a high rating, the model can recommend the user to watch that movie.

(In reality, rather than performing KNN on pairs of movies at a time, a multidimensional approach would be taken, comparing ratings of many movies at once.)

See also

Reference the scikit-learn documentation for supervised k-nearest neighbours classification by searching online for scikit-learn's section 1.6.2 "Nearest Neighbors Classification" (<https://scikit-learn.org/stable/modules/neighbors.html#nearest-neighbors-classification>).

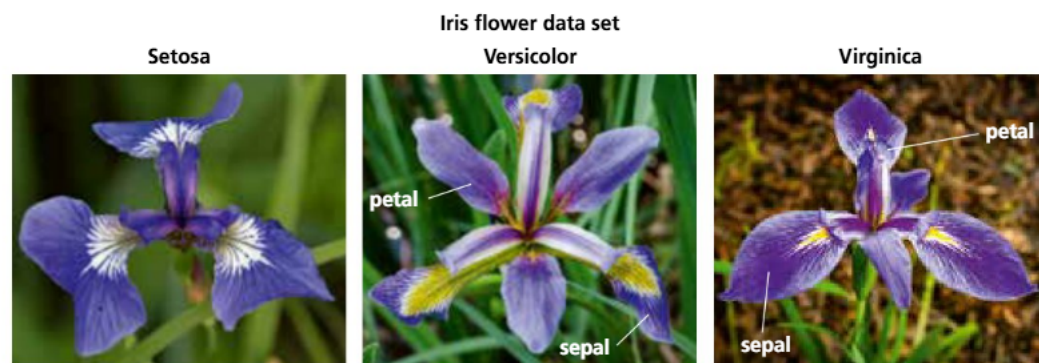
Decision trees

While KNN is considered a lazy learner (it does very little during the training phase and defers most of the computation until prediction), **decision trees** are considered eager learners that build a classification model during training.

Conceptually, you can think of decision trees as a large flowchart or series of nested *if-else* statements that are used to determine classification. Rather than having to manually determine the decision points and write programming code for the *if-else* statements yourself, the algorithm will analyse the training data to automatically determine the cutoff values for each decision point along the way, and how deep to make the nested tree.

Decision trees make for an easy-to-maintain algorithm since the model can be retrained and its decision paths and threshold values subsequently adjusted based on new data. Decision-tree algorithms are a scalable solution that works with large and complex data sets compared to the impracticalities associated with maintaining *if-else* statements yourself that might have hundreds or thousands of decision points and pathways, and therefore also be very much prone to human error.

◆ **Decision tree:** a graphical representation of conditions that will result in a classification decision being made; think of it as a decision-making flowchart that the machine learning model creates.

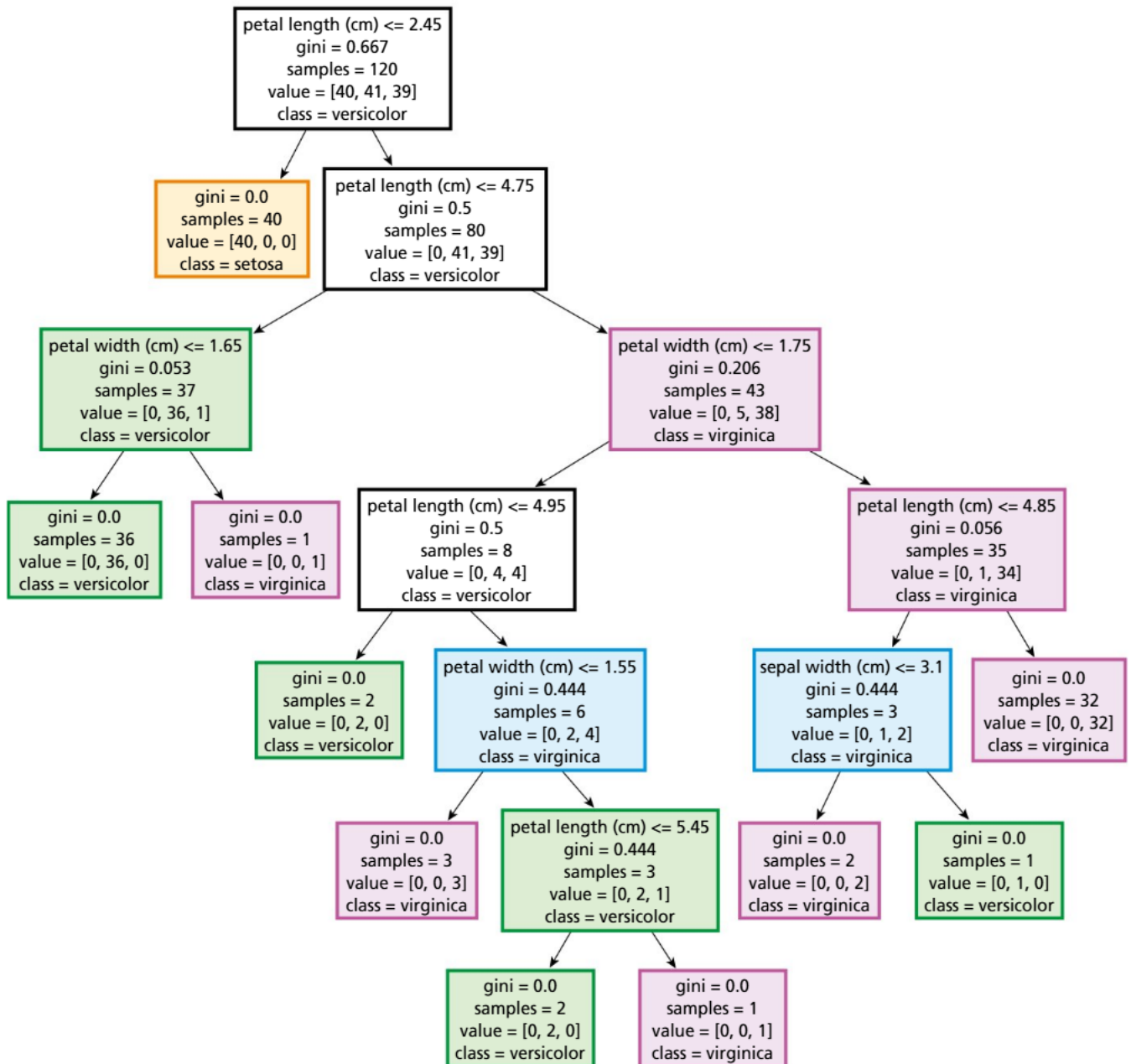


■ Iris flowers

The iris data set is commonly used as an introduction to decision trees. It contains measurements of 150 irises (a type of flower), one-third each of setosa, versicolor and virginica. For each of the 150 measurements, there are four features (variables):

- sepal length in cm
- sepal width in cm
- petal length in cm
- petal width in cm.

One version of the final trained decision tree might look like this:



■ Trained decision tree for the iris data set

Starting at the top of the tree, observe the first decision is whether the petal length is ≤ 2.45 cm. The other values printed in the node advise about the model's prediction if there was no further processing beyond this point. The significance of the terms used are:

- **Gini** indicates the decision tree would be 67 per cent uncertain in the prediction generated (which makes sense as, without taking any branch in the tree, it will effectively be making a 1-in-3 guess).
- **Samples** indicates that all 120 samples passed through this node (note it is 120 instead of 150 as, when this diagram was produced, 30 samples were retained as unseen for validation testing purposes).

- **Value** indicates the spread of the three classifications at this point (40 setosa, 41 versicolor and 39 virginica).
- **Class** indicates that the prediction at this point would be versicolor (since it had the most samples, with 41).

Based on the measurement of petal length, you either take the branch to the left, if the petal length is ≤ 2.45 cm, or to the right, if the petal length is > 2.45 cm.

Keep traversing the tree until you reach a termination point, or you have gone as deep into the tree as you would like and wish to terminate, obtaining the model's best prediction at that point. (Keep in mind that, while the model represented here only has a maximum depth of six levels, more complex decision trees can easily have maximum depths of hundreds or thousands of layers, hence the option for stopping once a particular depth is reached.)

The following Python will implement the iris problem.

Python

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
# Load the Iris data set
iris_df = pd.read_csv("iris.csv")
x = iris_df.drop("species", axis=1) # features are all the columns except species
y = iris_df["species"] # target variable
# Split the data set into a training set and a test set
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=42)
# Create a decision tree and generate predictive data
clf = DecisionTreeClassifier(max_depth=10, random_state=42)
clf.fit(x_train, y_train)
y_pred = clf.predict(x_test)
# Plot the decision tree
plt.figure(figsize=(12, 8))
tree.plot_tree(clf, filled=True, feature_names=iris.feature_names, class_names=
iris.target_names)
plt.show()
# Validate accuracy using the test data
accuracy = clf.score(x_test, y_test)
print(f"Accuracy of the decision tree classifier is: {accuracy:.2f}")
# Generate a prediction for a manual data point
new_flower_measurements = np.array([[5.0, 3.5, 1.5, 0.2]])
predicted_species = clf.predict(new_flower_measurements)
print(f"The predicted species for the new flower is: {predicted_species[0]}")
```

See also

Reference the scikit-learn documentation for decision trees by searching online for scikit-learn's section 1.10 "Decision Trees" (<https://scikit-learn.org/stable/modules/tree.html>).

The iris data set csv file can be downloaded from <https://github.com/paulbaumgarten/hodder-ibdp-computerscience>

One real-world application of decision trees is to assist with patient diagnosis. The features (variables) at issue in such a model may include demographic information (age, sex, height, weight); clinical measurements (blood pressure, glucose levels, haemoglobin, cholesterol); lifestyle factors (such as smoking status and alcohol consumption); along with symptoms, conditions, medical history and test results. It is easy to envisage such a model having dozens of features.

ACTIVITY

Thinking skills: Classification with KNN or decision trees?

Select a classification problem and perform a comparison analysis of both KNN and decision trees to solve the problem. Evaluate the trade-offs between the two approaches and create a decision matrix based on criteria such as ease of understanding, computational efficiency, performance on small vs large data sets, and so on.

Top tip!

When choosing between k-nearest neighbours (KNN), decision trees and artificial neural networks for a supervised learning classification scenario, consider the following:

- KNN suits moderate data sizes and low dimensions; decision trees handle mixed data sizes well; neural networks excel with large, complex data sets.
- KNN requires normalization; decision trees need minimal preprocessing; neural networks often require extensive preprocessing.
- KNN and decision trees are highly interpretable; neural networks are less so, and are often considered "black boxes".
- KNN are slow at prediction; decision trees offer fast predictions but can overfit easily; neural networks require significant computational power but handle non-linear data well.
- KNN easily integrate new data; decision trees and neural networks most often require retraining.

A4.3.3 Supervised learning: evaluation and tuning

Evaluation metrics

TOK

How can we know that current knowledge is an improvement upon past knowledge?

Machine learning requires evaluating the performance of new algorithms against benchmarks or previous models.

In the context of machine learning, improvement is often quantified in terms of performance on specific tasks. However, Theory of Knowledge invites you to question deeper aspects of this improvement: Does performing better on a task, like object recognition, necessarily mean the algorithm has gained more "knowledge"? Is the understanding deeper or merely more functional?

Additionally, metrics can sometimes be misleading. For example, an algorithm might score very highly on accuracy but fail in particular scenarios that weren't well represented in the training data.

Within supervised learning, there are several important, established metrics that can be used to evaluate the effectiveness of your model.

◆ **Confusion matrix:** a simple pictorial means of representing how well a machine learning model is performing.

The starting point would typically be to produce a **confusion matrix**. For a binary classification problem, the data forms a two-row, two-column table modelled as follows:

		Predicted	
		Predicted positive	Predicted negative
Actual	Actually positive	True positive	False negative
	Actually negative	False positive	True negative

The number of scores that are *true positive*, *false negative*, *false positive* or *true negative* are written into the respective cell (and typically colour coded with dark shading to indicate higher quantities), the idea being that it is a quick visual indicator of the success of your model. If the highest numbers (and dark shading) run down the diagonal of *true positive* and *true negative*, then that is a good sign.

A confusion matrix can also be produced for higher dimensional classification problems. In that instance, each possible classification would be turned into rows and columns. In this case, the diagonal set of cells from top left to bottom right would again represent correct predictions.

Using your confusion matrix, you can proceed to calculate the accuracy, precision, recall and F1 scores.

- **Accuracy:** The fraction or ratio of correct predictions

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

- **Precision:** The fraction or ratio of correct positive predictions to total positive predictions
For instance, of all images recognized as being “cats”, how many of them were correctly classified? Alternatively, of all the “spam” predictions, how many of those were correctly “spam”? This is important when a false positive may have a significant consequence.

$$\text{precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

- **Recall:** The fraction or ratio of correct positive predictions to actual positives

For instance, this could be number of patients correctly predicted to have diabetes, out of all the patients who truly have diabetes.

$$\text{recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

- **F1 score:** The harmonic-mean of precision and recall; it is particularly useful where both false positives and false negatives may carry significant consequence

A harmonic-mean is different from an arithmetic mean in that it will always give a score closer to the smaller of the two numbers. The F1 score will range from 0 to 1, with 1 being the best score.

$$\text{F1 score} = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

Consider using an F1 score in the criminal justice system, where an algorithm has been devised to predict whether an individual will re-offend if released on parole. (Note that there have been real-world problems in using machine learning modes for this exact scenario; refer to A4.4 Ethical considerations for more.) In this situation, precision measures the correctness of positive predictions. High precision means most individuals predicted to re-offend actually did re-offend. Recall measures how well the model successfully identifies those who will re-offend, so that re-offenders are not being ignored by the system. Both are important for matters of public safety, so combining them through the use of the F1 score is valuable.

Another example scenario is to imagine a school using face recognition to automatically record attendance as students walk through the school gate. In this scenario, high precision implies that when the system identifies a student, it is identifying the correct student (rather than recording the wrong student as present), and high recall suggests the system correctly identifies most or all students who pass through the gate.

Common mistake

It is common for students to over-rely on accuracy and neglect the nuance provided by the other metrics. Have a clear understanding of the distinct roles of precision and recall.

- **Accuracy** is the overall correctness of the model (both true positives and true negatives).
- **Precision** is the proportion of positive identifications that were actually correct (important when the cost of a false positive is high).
- **Recall** is the proportion of actual positives correctly identified (important when the cost of a false negative is high).
- **F1** is the harmonic-mean of precision and recall (useful when a balance between precision and recall is needed).

◆ **Hyperparameter:** a parameter (or value assigned to a variable) that is set before the learning process, which guides the algorithm as it learns.

Hyperparameter tuning

Hyperparameters is the technical term for the global variables that affect the entire model.

Commonly used hyperparameters include:

- learning rate (neural networks)
- activation function (neural networks)
- number of hidden layers (neural networks)
- maximum depth of tree (decision trees)
- number of neighbours (k-nearest neighbours)
- number of clusters (unsupervised clustering)
- other variables as required by the model.

Hyperparameter tuning is the process of experimentation and adjustment of the combination of parameters that results in optimal performance of a model.

Key information

Hyperparameters exist in all types of machine learning, not just in supervised learning. Take the time to identify the hyperparameters in whatever algorithm you are using and the effect their adjustment will have.

Common mistake

Avoid creating models that are too complex for your data; simpler models are easier to understand and debug, and often perform better on new, unseen data.

Overfitting and underfitting

Overfitting occurs when the model effectively memorizes detail from the training data that is too fine grained for it to make sufficient generalizations for use on unseen data. Reducing the depth of a decision tree, increasing the regularization strength in a linear model or reducing the number of neurons in hidden layers may help with this problem.

Underfitting occurs when the model is too simple and hasn't learned enough detail about the underlying patterns involved, such that the model also performs poorly on unseen data.

Signs of overfitting include:

- the model performs significantly better on the training data compared to the validation data
- a complex architecture is used, with many features

- the training error rate decreases, but the testing error rate increases after a given number of epochs
- reducing the model's complexity improves test performance.

Signs of underfitting include:

- the model performs poorly on both the training and test data sets
- a simple model is used, with minimal features
- there are insufficient features to adequately capture the characteristics of the data
- increasing complexity or adding features improves test performance.

A4.3.4 Unsupervised learning: clustering techniques

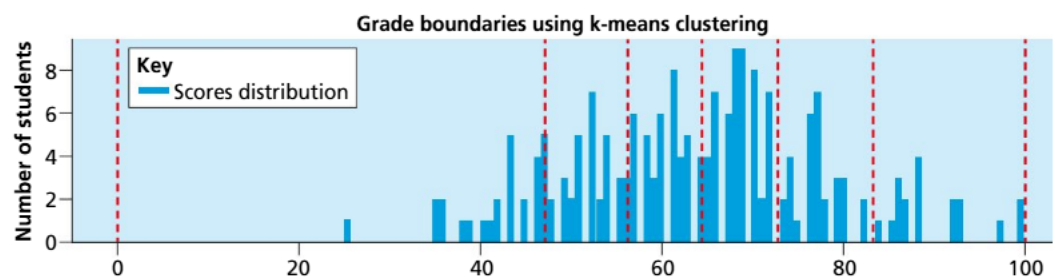
To review, unsupervised learning is where the data set your model is trained on is unlabelled. That is, you don't supply the *correct* answer that corresponds with each datum you supply. Rather than looking for data that is similar to known answer values, the features of the unlabelled data are compared for similarities among them all, with the goal of identifying naturally forming clusters in the groupings of data.

■ K-means clustering

K-nearest neighbours is very commonly used for unsupervised **clustering**, in addition to the supervised learning approaches already considered.

With the supervised approach, when a new, unlabelled data point is introduced, the algorithm measures the distance (often Euclidean) from the new point to all those in the training set. From there, it identifies the nearest neighbours to assign a predictive label to the new value. With the unsupervised approach, when a new data point is introduced, the algorithm similarly measures distances to other data points to determine its nearest neighbours but, instead of predicting a label, it uses these relationships to identify the groupings, or clusters, within the entire data set.

The main weakness with a KNN approach is that it assumes clusters are spherical and of similar size, making it sensitive to initial centroids and outliers.

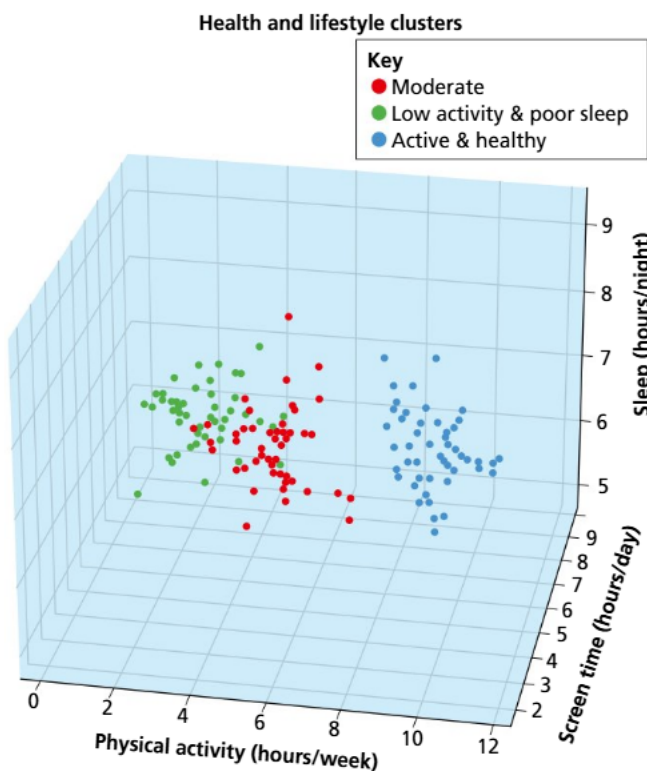
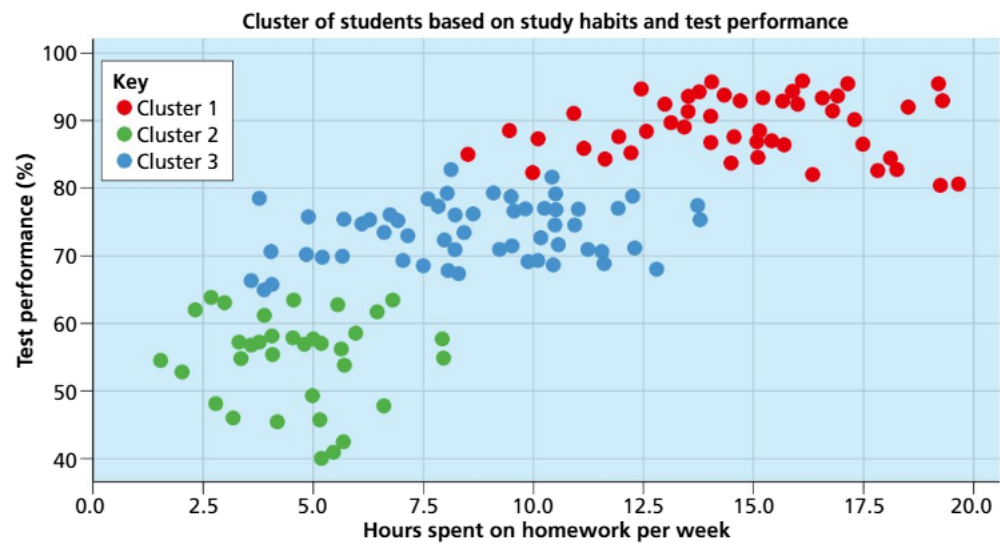


- K-means clustering with one dimension

In this example, k-means clustering has been used to determine grade boundaries for a cohort of 200 students. Asked to cluster the grades into six buckets, one for each letter grade, the algorithm determined the following boundaries:

- A: 83.48 to 100
- B: 72.74 to 83.48
- C: 64.45 to 72.74
- D: 56.30 to 64.45
- E: 47.09 to 56.30
- F: 0 to 47.09

Moving from one to two dimensions makes the identification of clusters more accurate. In this instance, k-means could look like the following:



While providing a 3D visualization in the 2D medium of a textbook is problematic, hopefully the point still gets across with the following illustration that, as dimensions go up, so the identification of clusters becomes easier, given the data points become further spread out. This works provided there is still enough data to form actual clusters, otherwise the curse of dimensionality will soon kick in (as can be seen in the illustration; there are many cells with no value).

Python

```
# Example implementation of k-means clustering
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Generate synthetic data
```

```

np.random.seed(42)
# Students who spend little time but perform variably
group1 = np.random.normal(loc=[5, 60], scale=[2, 10], size=(50, 2))
# Students who spend a moderate amount of time and perform moderately
group2 = np.random.normal(loc=[10, 75], scale=[2, 5], size=(50, 2))
# Students who spend a lot of time and perform well
group3 = np.random.normal(loc=[15, 90], scale=[2, 5], size=(50, 2))
# Combine the groups into a single data set
data = np.vstack([group1, group2, group3])
# Apply k-means clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(data)
labels = kmeans.labels_
# Plot results
plt.figure(figsize=(10, 6))
colors = ["red", "green", "blue"]
for i in range(3):
    plt.scatter(data[labels == i, 0], data[labels == i, 1], color=colors[i],
                label=f"Cluster {i+1}")
plt.title("Cluster of Students Based on Study Habits and Test Performance")
plt.xlabel("Hours Spent on Homework per Week")
plt.ylabel("Test Performance (%)")
plt.legend()
plt.grid(True)
plt.show()

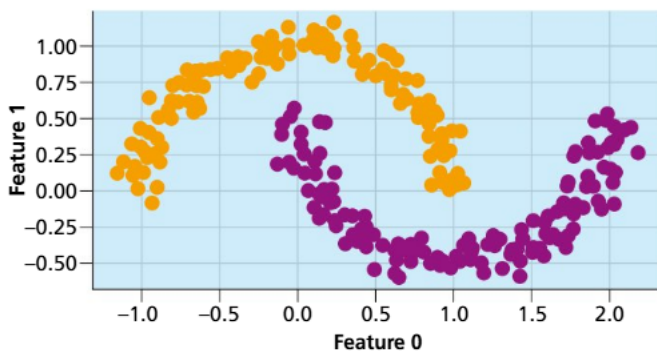
```

Common mistake

Assuming clusters are globular; k-means does not work well with non-spherical clusters.

See also

Reference the scikit-learn documentation for k-means clustering by searching online for scikit-learn's section 2.3.2 "K-means" (<https://scikit-learn.org/stable/modules/clustering.html#k-means>).



■ Spectral clustering

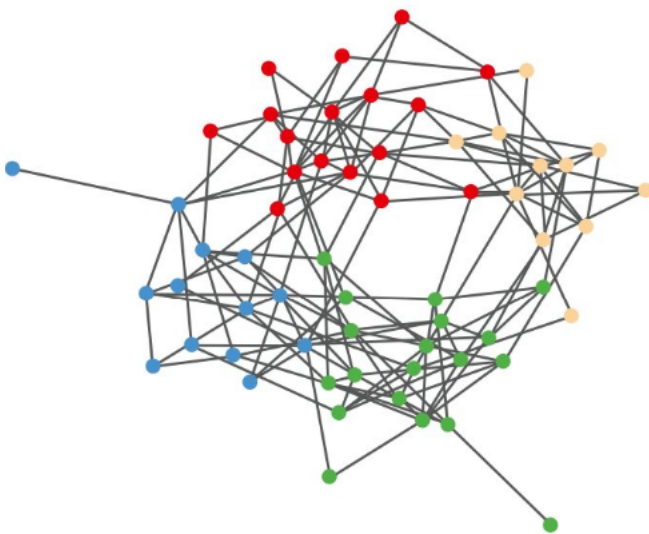
Spectral clustering

Spectral clustering is another technique that is useful where clusters are not linearly separable. To classify any new data point, it will look at where the new point fits best among the groups already made, like finding which circle of friends a new student would fit into at school.

Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.neighbors import kneighbors_graph
from sklearn.cluster import SpectralClustering
# Generate synthetic data (two interleaving half circles)
x, _ = make_moons(n_samples=300, noise=0.07, random_state=42)
# Create a k-nearest neighbours graph
knn_graph = kneighbors_graph(x, n_neighbors=10, include_self=False,
                             mode="distance")
# Apply spectral clustering using the KNN graph
spectral = SpectralClustering(n_clusters=2, affinity="precomputed",
                              assign_labels="kmeans", random_state=42)
labels = spectral.fit_predict(knn_graph)
# Plot the results
plt.figure(figsize=(8, 4))
plt.scatter(x[:, 0], x[:, 1], c=labels, cmap=plt.cm.rainbow, edgecolor="k", s=50)
plt.title("Spectral Clustering results")
plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
plt.show()
```

Social network analysis using spectral clustering



■ Spectral clustering can group people according to social networks

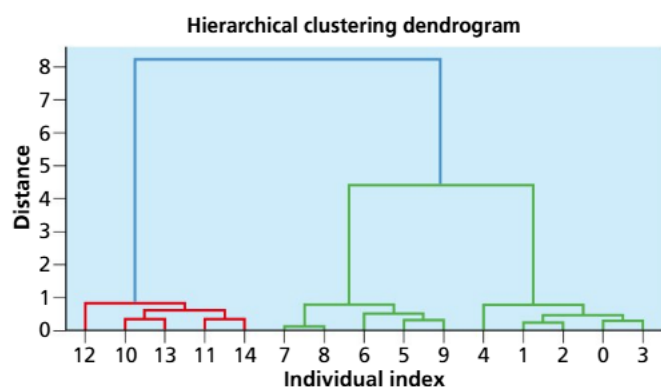
Spectral clustering may be useful in contexts such as social network analysis to identify communities within networks by treating nodes as people and edges as their relationships.

See also

Reference the scikit-learn documentation for spectral clustering by searching online for scikit-learn's section 2.3.5 "Spectral clustering" (<https://scikit-learn.org/stable/modules/clustering.html#spectral-clustering>).

Hierarchical clustering

This approach builds a tree of clusters and doesn't require the number of clusters to be specified in advance. It provides a dendrogram (a tree-like diagram) to interpret the data by viewing at different levels of granularity; however, it is computationally intensive for large data sets.



■ Hierarchical clustering tree

Example applications include genealogy research to analyse genetics to understand family relationships, and organizing library resources such as books and journals in a manner that reflects similarity of content based on topics, themes or authors.

In this plot of a family tree, individuals from the same family are grouped closer together first and, as you move up the dendrogram, families start merging based on their similarities (distances).

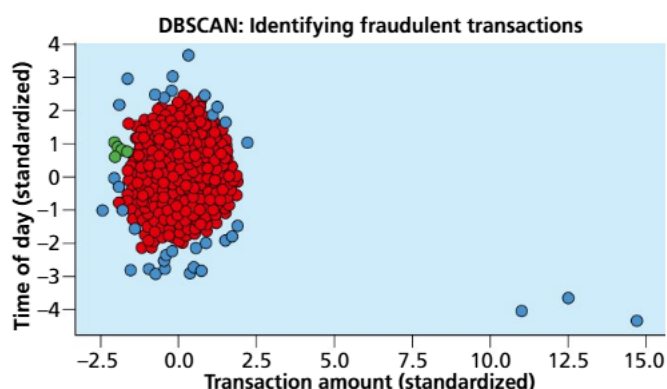
See also

Reference the scikit-learn documentation for hierarchical clustering with a dendrogram by searching online for scikit-learn's "Plot Hierarchical Clustering Dendrogram" (https://scikit-learn.org/stable/auto_examples/cluster/plot_aggglomerative_dendrogram.html).

DBSCAN clustering

Key information

DBSCAN stands for Density-Based Spatial Clustering of Applications with Noise.



■ Density-based spatial clustering

DBSCAN clustering will group together points that are close to each other based on a distance measurement and a minimum number of points. It is very effective for data with clusters of similar density. Unlike k-means, DBSCAN does not require the number of clusters to be specified. It can find arbitrarily shaped clusters and can handle noise and outliers.

An example application might be to detect fraudulent financial transactions by clustering on similarities of amount, location and time. DBSCAN can identify the dense clusters that are "typical" and then separate out unusual transactions for alerts.

See also

Reference the scikit-learn documentation for DBSCAN by searching online for scikit-learn's section 2.3.7 "DBSCAN" (<https://scikit-learn.org/stable/modules/clustering.html#dbscan>).

◆ Association rule:

a process of finding patterns of co-occurrence in data; this means, given the presence of one item in a record, how likely it is that another item will be present.

A4.3.5 Unsupervised learning: association rule

The **association rule** can be understood as a data mining technique that seeks to find co-occurrences within a data set. It is another form of unsupervised learning. The technique is commonly applied for market analysis, as well as crime analysis, healthcare, web / app behaviour tracking, and more.

There are three key metrics associated with association rule learning:

- **Support:** The proportion of transactions that include a particular item or combination of items.
- **Confidence:** The likelihood of occurrence of a particular item (B) when given some other item (A).
- **Lift:** The degree to which two items will appear together in this model, compared to the expected likelihood of them appearing together if the items were statistically independent; a lift value greater than 1 indicates the presence of item A increases the likelihood of B appearing.

The following example uses a data set of transactions from a fresh food market to find the items that are frequently purchased together. You can download the data set from <https://github.com/paulbaumgarten/hodder-ibdp-computerscience>

The first table, *frequent itemsets*, shows the support value for each set of items. In this case, 84 per cent of transactions include the sale of Milk. The *association rules* table illustrates the association between the antecedent (prerequisite) item and the consequent (resulting) item. In this case, the table shows:

- 62 per cent of transactions involve both Milk and Bread
- there is a 73 per cent likelihood that the customer will also purchase Bread if they purchase Milk
- the lift of 1.05 indicates that Bread is 1.05 times more likely to be purchased with Milk than without it.

Frequent Itemsets:					
	support	itemsets			
0	0.84	(Milk)			
1	0.70	(Bread)			
2	0.60	(Butter)			
3	0.60	(Egg)			
4	0.54	(Cheese)			
..	...	...			
244	0.40	(Pasta, Butter, Bacon, Chicken)			
245	0.40	(Apple, Banana, Coffee, Chicken)			
246	0.40	(Apple, Banana, Bacon, Coffee)			
247	0.42	(Banana, Bacon, Coffee, Chicken)			
248	0.40	(Pasta, Banana, Bacon, Chicken)			

Association Rules:					
	antecedents	consequents	support	confidence	lift
0	(Milk)	(Bread)	0.62	0.738095	1.054422
1	(Bread)	(Milk)	0.62	0.885714	1.054422
2	(Butter)	(Milk)	0.50	0.833333	0.992063
3	(Egg)	(Milk)	0.50	0.833333	0.992063
4	(Cheese)	(Milk)	0.50	0.925926	1.102293
..	...	...	...	...	...
821	(Pasta, Bacon)	(Banana, Chicken)	0.40	0.833333	1.602564
822	(Pasta, Chicken)	(Banana, Bacon)	0.40	0.740741	1.322751
823	(Banana, Bacon)	(Pasta, Chicken)	0.40	0.714286	1.322751
824	(Banana, Chicken)	(Pasta, Bacon)	0.40	0.769231	1.602564
825	(Bacon, Chicken)	(Pasta, Banana)	0.40	0.714286	1.552795

Python

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
# Load data set
df = pd.read_csv("biased_transactions.csv")
# Generate frequent itemsets
frequent_itemsets = apriori(df, min_support=0.4, use_colnames=True)
# Generate association rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
# Display the results
print("Frequent Itemsets:")
print(frequent_itemsets)
print("Association Rules:")
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]])
```

This example uses an algorithm called Apriori in the `mlxtend` library for discovering the frequent data sets. Here is a high-level overview of the Apriori algorithm:

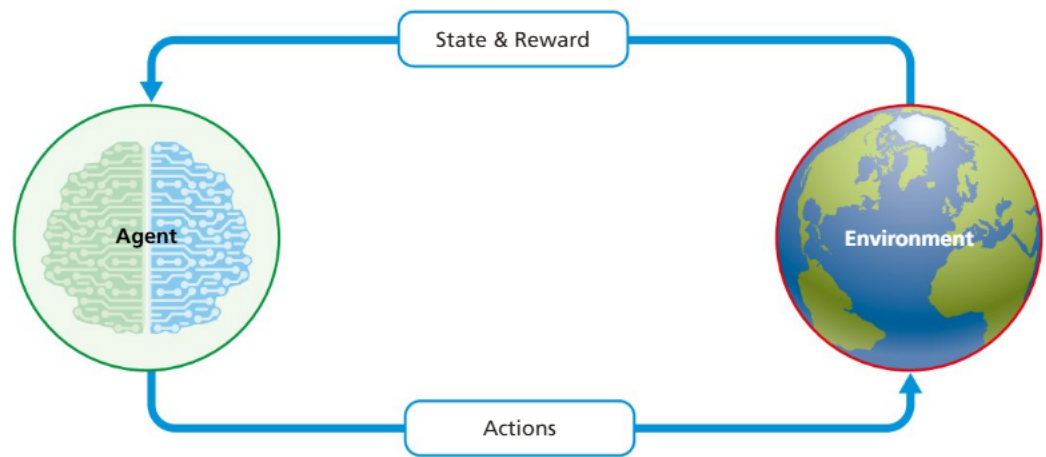
- Determine a threshold for the minimum level of support that will be considered (40 per cent in the code example above).
- Identify individual items in the data set that meet the threshold and store their appearance count.
- Progress to identifying pairs of items and larger groupings of items in the data set that also meet the minimum threshold.
- If a set of items does not meet the threshold at a small level, it can be removed from further consideration.
- The result is all combinations of items that appear frequently together at or above the minimum threshold.

See also

Reference the `mlxtend` documentation for association rule processing by searching online for `mlxtend` documentation for association rule processing (https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules).

A4.3.6 Reinforcement learning

Previously, “reinforcement learning” was described as learning from trial and error. In that vein, it can be likened to a toddler learning to walk. Every time the toddler falls over, they learn a little more about how to correctly balance themselves next time, until they eventually become a stable and confident walker.



■ Reinforcement learning flowchart

As with most machine learning algorithms, reinforcement learning introduces some new terminology to consider:

- **Agent:** The machine learning model that makes the decisions on what to do.
- **Environment:** The world, as perceived by the agent.
- **State:** A snapshot in time of the world. State is the data that communicates the current situation or the environment. Careful consideration of your state data is critical when developing a reinforcement learning algorithm. What data will you provide to the agent to help it learn the task you have for it? For instance, when training an agent to play a snake game, do you give it values indicating the distance and bearing of the apple, or a pixel map of the entire world?
- **Action:** An operation or behaviour that the agent can perform in the environment (for example walk forward, turn left, turn right).
- **Reward:** An immediate return from the environment in response to the agent's action. Reward may be positive or negative (a punishment).
- **Policies:** The strategies the agent will use to map states to actions. Think of policies as the agent's mental if-this-then-that list.

The general process is as follows:

- The agent will typically begin with a randomized policy, as it has no existing knowledge of the environment.
- The agent observes the environment. Based on what it perceives, and the policy it has recorded so far, the agent chooses an action to perform.
- The agent performs the selected action.
- The environment updates to a new state.
- The environment provides feedback via a reward to the agent.
- The agent updates its policy based on the reward feedback received.
- The process repeats.

Reinforcement learning usually involves a combination of **exploration** and **exploitation**. "Exploration" is when the agent ignores its learned policy and tries something new.

“Exploitation” is when the agent follows the learned policy and behaves according to what it learned. Typically, an algorithm will start with a heavy emphasis on exploration, as the algorithm hasn’t had much opportunity to learn anything yet. Over time, the hyperparameter for the exploration / exploitation ratio, known as the “learning rate”, should adjust so as to start deferring to the learned data more.

■ Q-learning

The agent’s policies are responsible for maintaining what the agent has learned. While there are a few approaches to this, one of the most common is known as “Q-learning”.

Q-learning can be thought of as using a 2D array or other data structure to create a giant lookup table for every possible state and action combination. It stores a value for each possible permutation of the two to predict what reward it would receive in each scenario.

	Action 1	Action 2	Action 3	Action 4
State 1	-50	0	10	0
State 2	10	20	0	10
State 3	0	-10	0	50

The table illustrates a simplified version of a Q-learning 2D array. The data would suggest:

- When state 1 is seen, the best thing the agent can do is action 3, and it should avoid doing action 1.
- When state 2 is seen, the agent should do action 2, but action 1 and 4 would also give it a reward.
- When state 3 is seen, the agent should do action 4 for a large reward (possibly winning the game), and avoid action 2.

A Q-learning table can be very large, given the array size is determined by all possible states and all possible resulting actions. When the data requirements are unfeasibly large, an alternative approach is to use an artificial neural network to learn generalizations about the state and resulting output actions. When a neural network is used, it is known as a “Deep Q-Network”.

Here is a pseudocode overview of the process:

```
Initialize the Q-table with all zeros (or some initial values)
for each round:
    Initialize the state S to the starting point of the game
    while the episode is not finished:
        Choose action A from state S using a policy derived from Q
        Take action A
        Observe the immediate reward R and the next state S'
        Update the Q-table value for the original state S and action A:
             $Q(S, A) \leftarrow Q(S, A) + \alpha * (R + \gamma * \max_{all\_actions} Q(S', all\_actions) - Q(S, A))$ 
        S <- S' # Move to the next state
    end while
end for
```

There are a few comments to note about this pseudocode:

- When choosing action A, take into consideration whether the algorithm should exploit its Q-table or explore other alternatives.
- Alpha here refers to the learning rate. A higher alpha means that newer information has a greater impact on updating the Q-values, allowing the agent to adapt quickly to changes in the environment. A lower alpha will cause slower updates, making the agent more stable but also slower to learn. A starting value between 0.01 and 0.05 would be normal.
- Gamma here refers to the discount factor for how much future anticipated rewards should be considered when making a decision. The ultimate goal of most scenarios is to find an optimal policy that provides the maximum cumulative reward. A gamma value close to 0 makes the agent “myopic” (short-sighted), heavily prioritizing immediate rewards. Conversely, a gamma close to 1 encourages the agent to consider future rewards more strongly, valuing them almost as much as immediate rewards. This makes the agent “far-sighted”, planning over a longer horizon.
- This line in the pseudocode is known as the “Bellman equation”:

$$Q(S, A) \leftarrow Q(S, A) + \alpha * (R + \gamma * \max_{all_actions} (Q(S', all_actions)) - Q(S, A))$$

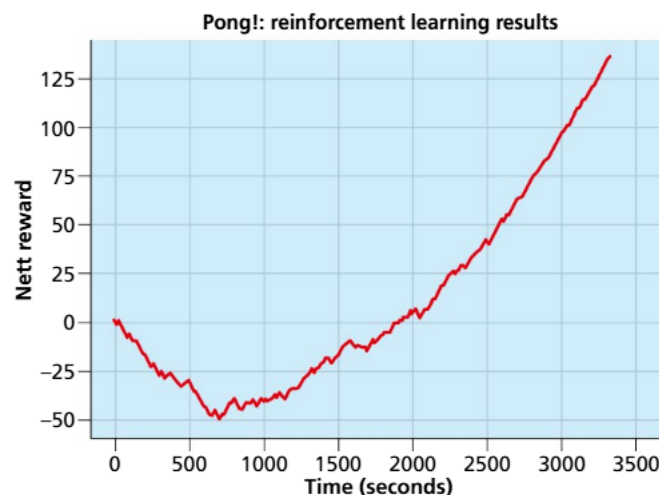
and it states that the Q-value for a state-action is equal to the immediate reward plus the discounted value of the best action to take in the next state, adjusted for the learning rate.

■ Example: Pong! game

The following example is a Pong!-style paddle-and-bouncing-ball game. It uses the *pygame-ce* library for the graphics, and a *numpy* array for the Q-table. As can be seen in the results chart, this agent requires about 20 minutes of training before it begins showing acceptable results.



■ Pong! game



■ Pong! game: nett reward over time

Python

```
import pygame
import random
import numpy as np
import matplotlib.pyplot as plt
# Constants
```

```

WIDTH, HEIGHT = 200, 400
FPS = 30
PADDLE_WIDTH, PADDLE_HEIGHT = 30, 15
BALL_RADIUS = 7
BALL_COLOUR = (255, 255, 64)
PADDLE_COLOUR = (255, 64, 255)
BACKGROUND_COLOUR = (64, 64, 128)
# Initialize Pygame
pygame.init()
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()
class Paddle:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, PADDLE_WIDTH, PADDLE_HEIGHT)
    def move(self, x):
        self.rect.x += x
        self.rect.x = max(self.rect.x, 0)
        self.rect.x = min(self.rect.x, WIDTH - PADDLE_WIDTH)
    def draw(self):
        pygame.draw.rect(screen, PADDLE_COLOUR, self.rect)
class Ball:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, BALL_RADIUS*2, BALL_RADIUS*2)
        self.dx = random.choice([-4, 4])
        self.dy = random.choice([-4, 4])
    def move(self):
        self.rect.x += self.dx
        self.rect.y += self.dy
        if self.rect.top <= 0 or self.rect.bottom >= HEIGHT:
            self.dy = -self.dy
        if self.rect.left <= 0 or self.rect.right >= WIDTH:
            self.dx = -self.dx
    def draw(self):
        pygame.draw.ellipse(screen, BALL_COLOUR, self.rect)
# Game objects
paddle = Paddle(WIDTH//2 - PADDLE_WIDTH//2, HEIGHT - PADDLE_HEIGHT)
ball = Ball(WIDTH//2, HEIGHT//2)
# Q-learning parameters
LEARNING_RATE = 0.05
DISCOUNT_FACTOR = 0.99
epsilon = 0.1
# Let state have 10 positions for paddle and ball
# (reduces demands on Q-Table)
def get_state():
    paddle_mid = paddle.rect.x + PADDLE_WIDTH // 2
    ball_mid = ball.rect.x + BALL_RADIUS
    return (paddle_mid//20, ball_mid//20)

```



```

# Q-table
# 10 possible paddle positions, 10 ball positions, 3 actions
q_table = np.zeros((10, 10, 3))
def update_q_table(state, action, next_state, reward):
    # Bellman's equation
    old_value = q_table[state[0], state[1], action]
    next_max = np.max(q_table[next_state[0], next_state[1]])
    new_value = (1 - LEARNING_RATE) * old_value +
                LEARNING_RATE * (reward + DISCOUNT_FACTOR * next_max)
    q_table[state[0], state[1], action] = new_value
def choose_action(state):
    if random.random() < epsilon:
        # Explore: choose a random action
        return random.randint(0, 2)
    else:
        # Exploit: choose the best action from Q-table
        return np.argmax(q_table[state[0], state[1]])
def get_reward():
    if ball.rect.bottom >= HEIGHT:
        if ball.rect.colliderect(paddle.rect):
            return 1 # Reward for hitting the ball
        else:
            return -1 # Penalty for missing the ball
    return 0 # No reward or penalty
running = True
nett = 0 # nett reward
cumulative = [] # nett reward history for graphing
while running:
    screen.fill(BACKGROUND_COLOUR)
    for event in pygame.event.get():
        # Use the quit icon for game termination
        if event.type == pygame.QUIT:
            running = False
    # Agent decides and acts
    state = get_state()
    action = choose_action(state)
    if action == 1:
        paddle.move(-10) # Move left
    elif action == 2:
        paddle.move(10) # Move right
    # Update game state
    ball.move()
    paddle.draw()
    ball.draw()
    # Check reward & state, update Q table
    reward = get_reward()
    next_state = get_state()

```

```

update_q_table(state, action, next_state, reward)
if reward > 0:
    PADDLE_COLOUR = (0,255,0)
elif reward < 0:
    PADDLE_COLOUR = (255,0,0)
# Update nett reward for graph
nett += reward
cumulative.append(nett)
# Draw the game to screen
pygame.display.flip()
clock.tick(FPS)
pygame.quit()
# Graph results
time = [i/FPS for i in range(len(cumulative))]
plt.plot(time, cumulative)
plt.title("Pong: Reinforcement learning results")
plt.xlabel("Time (seconds)")
plt.ylabel("Nett reward")
plt.grid(True)
plt.show()

```

See also

Gymnasium is a popular library for learning about reinforcement learning. It provides a number of pre-written environments that you can experiment with: <https://gymnasium.farama.org>

A4.3.7 Genetic algorithms

◆ **Genetic algorithm:** imitates the concept of survival of the fittest and evolution by testing a population of possible solutions to a problem, using properties from the best-performing solutions to create a new population of possible solutions, and then repeating the process until a suitably performing solution has been identified.

Genetic algorithms are not normally classified within the traditional categories of “supervised learning”, “unsupervised learning” or “reinforcement learning”. They are considered an evolutionary algorithm. They learn through a process of optimization inspired by the process of natural selection.

A high-level overview of the algorithmic process is:

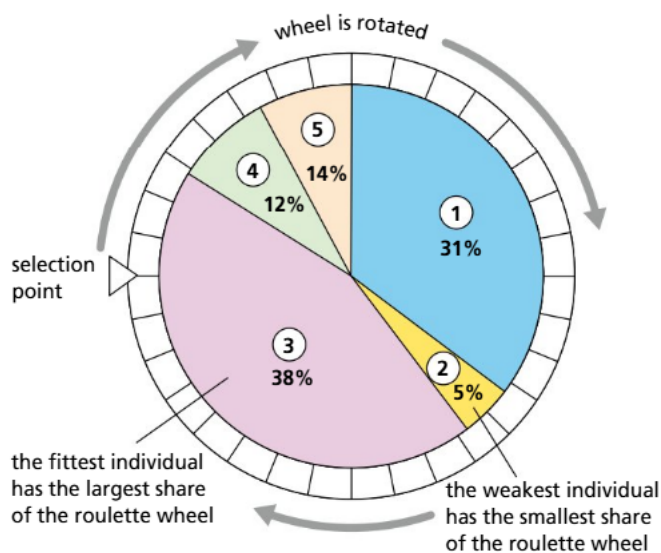
- Start with a *population* of possible solutions to the problem. This may be in the form of an array of strings, for instance, where each string is a randomly generated possible solution.
- Each item within the population is evaluated through a *fitness function* that returns a metric for how good the possible solution is.
- Pairs of solutions are *selected* for reproduction. Various algorithms will be discussed that make these selections, but generally the better the fitness function score, the more likely a solution is to be selected.
- The selected pairs then undergo *reproduction* using a *crossover* algorithm, where part of the genetic code of each parent is selected and then combined together to create a new possible solution.
- The new offspring may then undergo *mutation*. Random number functions are generally used so that only a small percentage of offspring undergo mutation, and then those selected have parts of their genetic code (their “solution” to the problem) randomly altered.

- Once a new generation of offspring has been generated, they become the current generation, and the process of calculating fitness, selection, reproduction and mutation repeats itself.
- The process repeats until whatever termination criteria you determine is satisfied. This might be to iterate for a given number of generations, or to iterate until a fitness score of a minimum threshold has been reached.

Genetic algorithms are used for problems where it is not essential to identify the most perfect, optimal solution, but where there is a degree of “close enough is good enough” flexibility.

Common example applications of genetic algorithms include:

- **Route planning**, such as the travelling salesperson problem. Consider the scenario of a salesperson who has 50 cities to visit. Rather than crisscrossing the countryside, genetic algorithms can find a *close-to-optimal* route to minimize the distance travelled. In the context of a long journey to 50 cities, it is not necessary to find the most perfect solution, so long as the solution is good enough. Put another way, if the algorithm can find a good solution in a few minutes, is it really worth hours or days of additional processing to find a solution that might be only 1 per cent better? At some point, the route is good enough to use.
- **Timetabling**, such as allocating students to their preferred classes where there are constraints on the number of teachers, rooms and classes available.
- **Civil and mechanical engineering** to help optimize design of structures such as bridges or buildings, and vehicle designs to make choices of materials based on durability, strength and cost.
- **Control systems and robotics** use genetic algorithms to optimize the controller for better stability and performance.
- **Finance applications** use genetic algorithms to help optimize a trade-off between risk and reward in selection of an investment portfolio.
- Within machine learning, a genetic algorithm can help select a subset of relevant features from a larger data set to improve model accuracy and reduce overfitting.
- In some types of machine learning problems, it is possible to use genetic algorithms for the training of an artificial neural network as an alternative approach to backpropagation (see later in this section).



■ Roulette wheel selection

■ Selection functions

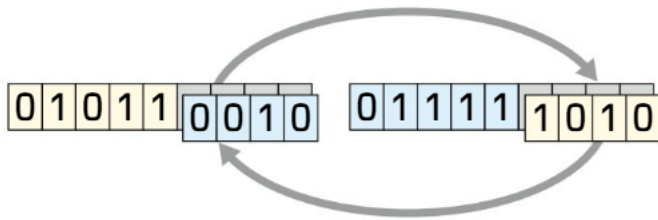
Selecting pairs of values for reproduction relies on the concept of weighted randomization. It is weighted in the sense that those with higher fitness scores should be more likely to be chosen for reproduction. Randomization is still important, however, to ensure that the algorithm doesn't become trapped in a local maximum.

One of the most commonly used approaches is the concept of a roulette wheel, where the portion of the wheel allocated is determined by the fitness function score.

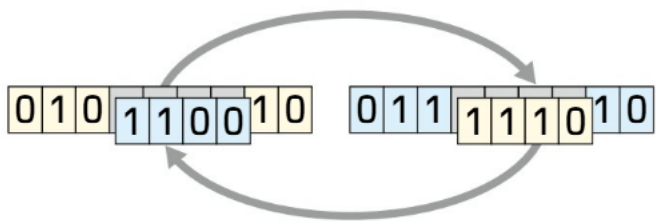
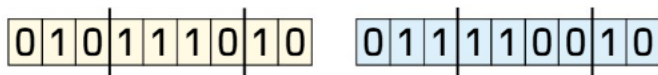
Crossover functions

Crossover functions define the algorithm used for reproduction – taking two input solutions and mixing them in such a way as to produce a “child”, representing a new valid solution.

There are a variety of common algorithms to do this, but each needs to be considered in the context of the problem. Any approach will likely need to be tweaked to ensure the “child” created through the process is valid for the scenario.



■ One-point crossover



■ Two-point crossover

Two common methods are the one-point crossover and the two-point crossover.

One-point crossover selects a random point in the gene sequence to slice the data. The data from up until the slice point is copied from parent 1 into child 1, and then the rest of child 1's data comes from parent 2. The inverse can also occur at the same time to create a second child.

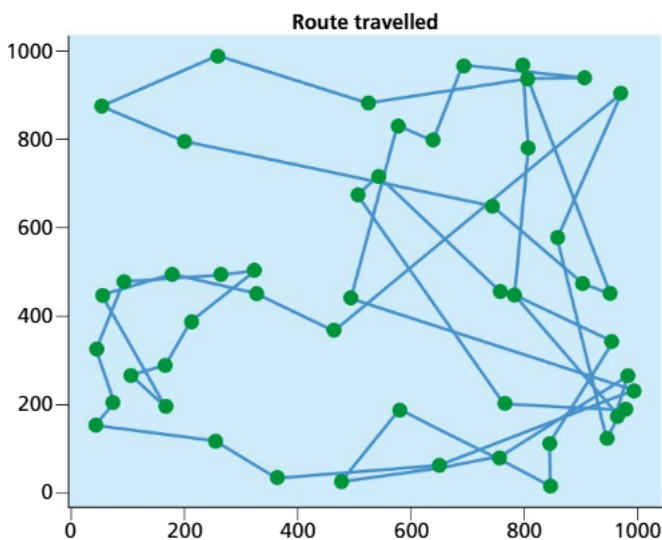
Two-point crossover works in a similar manner, except two slice points are selected.

There are other commonly used methods as well, but what matters most is that you are using a randomization function to create offspring that are a mix of the data from two parents.

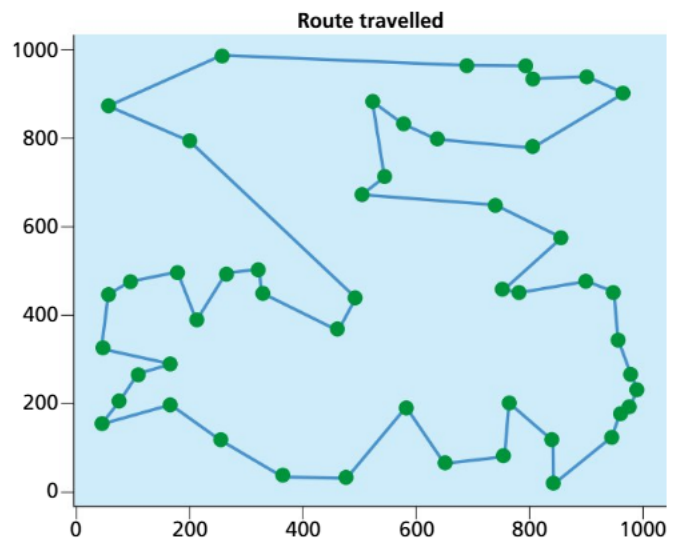
Top tip!

Choose appropriate genetic operators (selection, crossover, mutation) for your problem, and ensure diversity within the population to avoid premature convergence.

Example: Travelling salesperson



■ Travelling salesperson – random route



■ Travelling salesperson – optimized route

Consider the charts to represent a map of 50 cities that a travelling salesperson wishes to visit, with two possible routes for the journey. Clearly, the randomized journey is inefficient, whereas the optimized journey is a lot more efficient.

Python

```
def calc_distance(route):
    dist = 0
    for i in range(1, len(route)):
        # Get the co-ordinates for each pair of cities
        x1,y1 = coords[ route[i-1] ]
        x2,y2 = coords[ route[i] ]
        dist += math.sqrt( abs(x1-x2)**2 + abs(y1-y2)**2 )
    # Don't forget to return home at the end!
    x1,y1 = coords[ route[len(route)-1] ]
    x2,y2 = coords[ route[0] ]
    dist += math.sqrt( abs(x1-x2)**2 + abs(y1-y2)**2 )
    return dist
def calc_fitness(distances):
    max_distance = np.max(distances)
    min_distance = np.min(distances)
    normalized = (max_distance-distances)/(max_distance-min_distance)
    fitness = np.power(2.0, (normalized*100))
    return fitness
```

Step 3: Reproduction using a crossover method

In this case, the code below is using a modified form of one-point crossover. A point in the data is randomly selected, and the genetic data from parent 1 is copied across without change. After that, the remaining cities are copied over from parent 2 in the order in which they appeared in parent 2.

Here is a simple example with six cities:

PARENT 1 = [0, 4, 5, 1, 3, 2]

PARENT 2 = [4, 2, 0, 3, 5, 1]

Randomly decide how many genes to copy from parent 1. In this case, use three.

CHILD = [0, 4, 5, _, _, _]

Now copy the remaining values from parent 2, skipping the values already present from parent 1.

CHILD = [0, 4, 5, 2, 3, 1]

Python

```
def reproduce(parent1, parent2):
    # First set of genes will come from parent 1
    count_of_genes_from_parent1 = random.randint(0, len(parent1))
    child = parent1[0: count_of_genes_from_parent1]
    # Remaining genes will come from parent 2
    for i in range(0, len(parent2)):
        # Only include genes from parent2 not already provided by parent 1
        # (don't want to visit the same city twice)
        if parent2[i] not in child:
            child.append(parent2[i])
    return child
```

Step 4: Mutation

There is no fixed algorithm to use for mutation; it will vary somewhat depending on the context of your problem. In this case, the following code will, in 5 per cent of cases (the mutation rate hyperparameter), randomly pick a pair of cities in the route and swap them.

Python

```
def mutate(person, mutation_rate):
    if random.random() < mutation_rate:
        a = random.randint(0, len(person)-1)
        b = random.randint(0, len(person)-1)
        person[a], person[b] = person[b], person[a]
    return person
```

Step 5: Promote the children to be the active generation

This will be just a one-line task at the end of the loop to copy the data from the new generation that was being produced, into the array being used for the active generation.

Python

```
# Move to the next generation
population = next_generation
```

Step 6: Write the main function to bring it all together

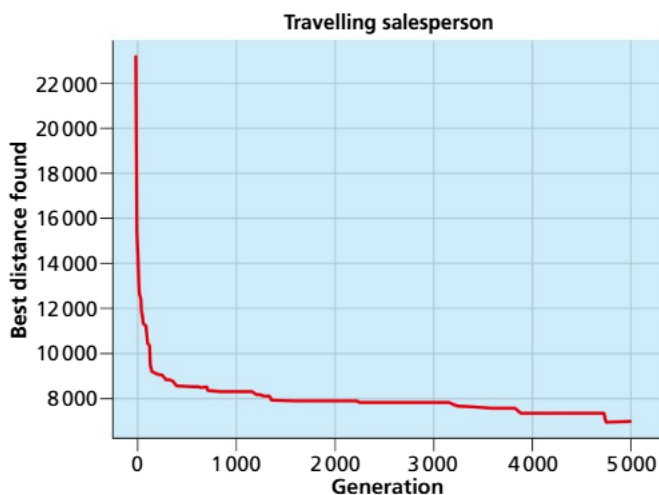
Python

```
import math, random, json
import numpy as np
with open("travelling-salesperson.json", "r") as f:
    coords = json.loads(f.read())
def travelling_salesperson(population_size=500, generations=5000):
    # Create randomized population
    population = create_initial_population(population_size)
    minimum = 50000
    generation_number = 0
    while minimum > 5000 and generation_number < generations:
        # Calculate fitness for each person
        generation_number += 1
        distances = np.array([calc_distance(population[n]) for n in range(0, population_size)])
        fitness = calc_fitness(distances)
        if generation_number % 25 == 0:
            print(f"Generation {generation_number}: Best {calc_distance(population[ np.argmax(fitness) ])} Mean { distances.mean() }")
        # Create the next generation
        next_generation = []
```

```

for p in range(0, population_size, 2):
    # Select parents
    # Select k=2 items from "population", using the values in "fitness"
    # to determine probability weighting.
    parents = random.choices(population, weights=fitness, k=2)
    # Reproduce
    child1 = reproduce(parents[0], parents[1])
    child2 = reproduce(parents[1], parents[0])
    # Mutate
    child1 = mutate(child1, 0.05)
    child2 = mutate(child2, 0.05)
    next_generation.append(child1)
    next_generation.append(child2)
    # Move to the next generation
    population = next_generation
# All done. What are the results?
fitness = np.array([calc_fitness(population[n]) for n in range
(0, population_size)])
best = np.argmax(fitness)
print(f"Best person after {generations} generations is #{best}")
print(f"Their travel distance: {calc_distance(population[best])}")
print(f"Their route: {population[best]}")
travelling_salesperson()

```



■ Best route found for travelling salesperson for each generation

As you let that execute, it will print an update every 25 generations with the progress it has made, similar to the following update:

```

Generation 25: Best 13496.55 Mean 14238.29
Generation 50: Best 11301.76 Mean 11541.97
Generation 75: Best 9265.94 Mean 9365.81
Generation 100: Best 8944.63 Mean 9033.31
Generation 125: Best 8727.39 Mean 8883.68

```

This chart shows the minimum travel distance calculated after each generation when the algorithm was executed by the author. There are a couple of important aspects to draw your attention to.

Firstly, and most obviously, is that most of the improvement in output occurs very quickly, after which the law of diminishing returns starts to apply.

The second thing to note, though, is that the output will frequently get stuck in a local minimum for several hundred generations before suddenly breaking out of it, so there can be benefits to gain by being patient enough to give that the opportunity to occur.

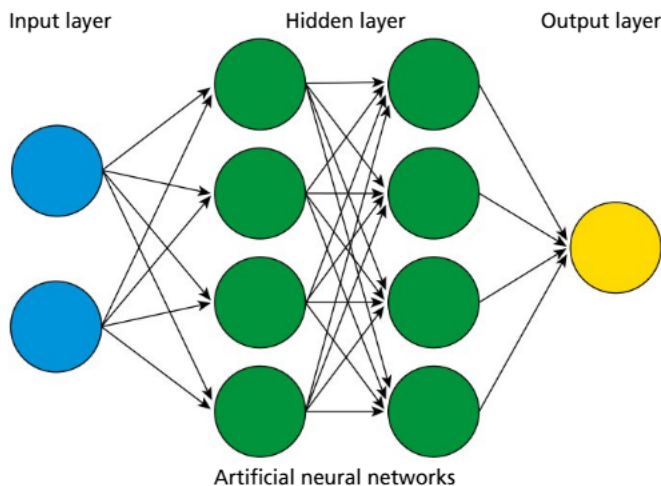
What is the lowest distance route you can obtain to visit all 50 cities?

Download the data file from <https://github.com/paulbaumgarten/hodder-ibdp-computerscience>

◆ **Perceptron:** the data structure at the heart of an artificial neural network; it represents a single artificial neuron that takes in multiple inputs and weights, and generates an output value.

A4.3.8 Artificial neural networks

An artificial neural network (ANN) is an algorithm that learns to make decisions by finding patterns in data using an approach modelled on the biological brain. Just as a biological brain consists of many neurons that are interconnected and send signals to each other via synapses, so too an artificial neural network is an algorithm that defines a series of nodes that transmit data between each other. These nodes are known as **perceptrons**, though they are also very commonly referred to as “neurons”.



■ Layers in an artificial neural network

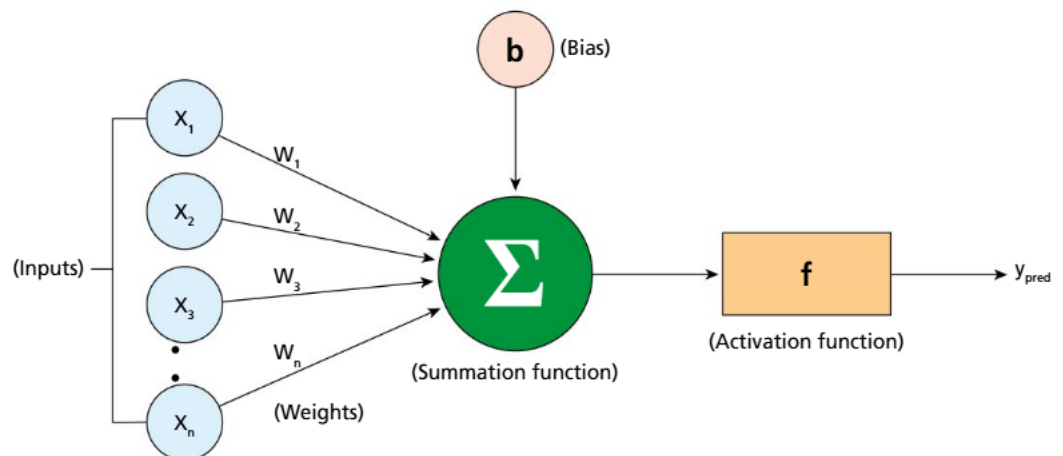
The diagram represents a typical structure for an ANN:

- The input layer receives the input values that the network is being asked to process. Each feature of your model requires its own input perceptron. The value given to each perceptron is numeric (either an integer or float, depending on the problem context).
- There are usually one or more hidden layers within an ANN. These are layers of perceptrons that identify patterns in the input data to make generalizations useful for the next layer. They receive the values from the previous layer, perform their calculations and then send their respective result to the next layer in the network.
- The output layer is where the ANN produces a final “answer” value or prediction.

The illustration in this case is also an example of a fully connected network, in that every perceptron from one layer is connected, and sends its output value to, every perceptron in the next layer. Fully connected networks are the norm.

■ A single perceptron

Zooming in on a single perceptron, the following sketch outlines the different elements at work:

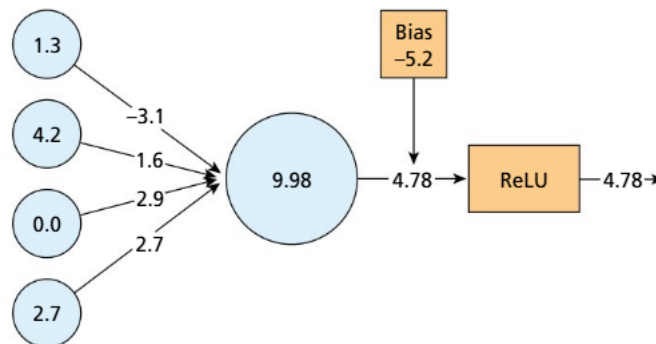


■ A perceptron in an artificial neural network

- **Input:** The perceptron receives an input value from every perceptron in the layer before.
- **Weight:** Every input has a weight associated with it. The weight is a value indicating the importance this particular neuron places on the values from the respective input. Weights are usually initialized with a random number between -1.0 and 1.0 and then adjusted by the training process.

- **Summation:** The product of each input value and its respective weight are summed together.
- **Bias:** Supplemental to weights, a typical neuron would also have a value called the “bias”. This is added to the value from the summation step, prior to using activation. The bias acts as a way to shift the decision boundary along the curve of the activation function. The bias is usually initialized with a random number between -1.0 and 1.0 and then adjusted by the training process.
- **Activation:** The activation function helps determine whether or not the neuron should be “active” (“inactive” in this case means the neuron would have an output value of 0). The activation function serves to introduce nonlinearity to make neurons more expressive. That is, it helps force the neuron to make a decision. For example, one commonly used activation function is ReLU, which results in a neuron being active for any positive value, and inactive (0) for any negative value. A comparison of common activation functions follows later in this section.
- **Output:** Finally, the resulting value returned from the activation function is sent onward to the neurons in the next layer, or the external system.

The following example is a walkthrough of the calculations for a perceptron:



■ **Example values in a perceptron**

- The perceptron receives input values of 1.3, 4.2, 0.0 and 2.7.
- Each input path has a weight of -3.1 , 1.6 , 2.9 and 2.7 respectively.
- Each input and its respective weight are multiplied, and the results added together:
 $(1.3 * -3.1) + (4.2 * 1.6) + (0.0 * 2.9) + (2.7 * 2.7) = 9.98$
- The bias value is added, which, for this example, is -5.2 :
 $9.98 + (-5.2) = 4.78$
- The resulting value is passed through the activation function, which, in this case, is ReLU:
 $ReLU(4.78) = 4.78$
- The output value 4.78 is passed along to the next layer in the network, or is given as the output value of the network, if it is the output layer.

From a mathematical perspective, up until the activation function, the rest of the perceptron can be considered a linear function, where the weights are the variable coefficients and the bias is the constant:

$$y = ReLU(x_1w_1 + x_2w_2 + x_3w_3 + x_4w_4 + b)$$

Or, to express it more generally:

$$y = activation\left(\left(\sum_{i=0}^n x_iw_i\right) + b\right)$$

Given the output of any individual perceptron can be expressed as a function, and that the inputs of perceptrons are either input values or the outputs of other perceptrons, it means that the entire artificial neural network behaves as a function.

◆ **Activation function:**

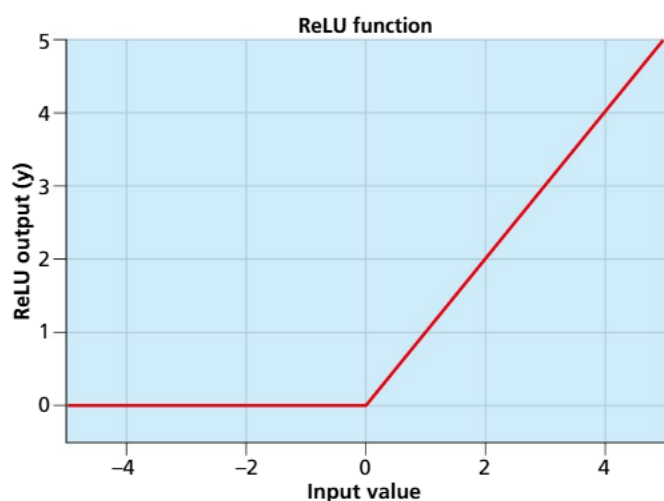
a mathematical function applied to the output of a neuron that is used to determine whether or not the neuron should be activated (considered to be “on”).

Common mistake

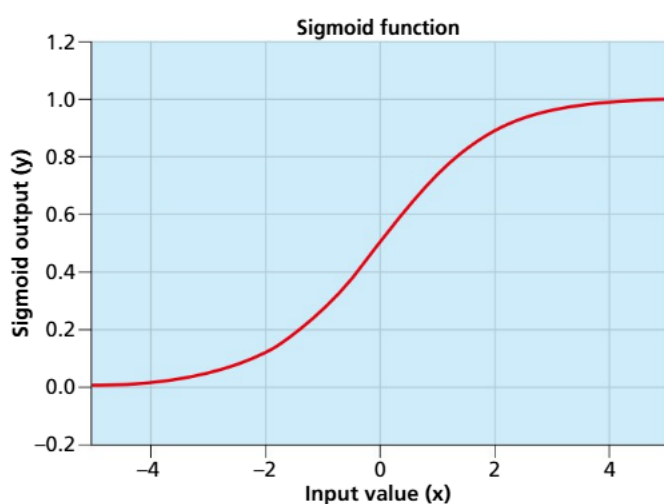
Overcomplicating the model architecture can lead to overfitting and high computational costs. Start with a simple architecture and gradually increase complexity, if necessary.

Activation functions

While there are a large variety of **activation functions** in use, there are four that are more common than all others: **ReLU**, **Sigmoid**, **Softmax** and **tanh**.



■ The ReLU function



■ The Sigmoid function

ReLU

ReLU (rectified linear unit) is often the default choice for ANNs. It is computationally efficient and is less likely to have a vanishing (approaching zero) gradient, unlike Sigmoid or tanh. The function for ReLU is:

$$f(x) = \max(0, x)$$

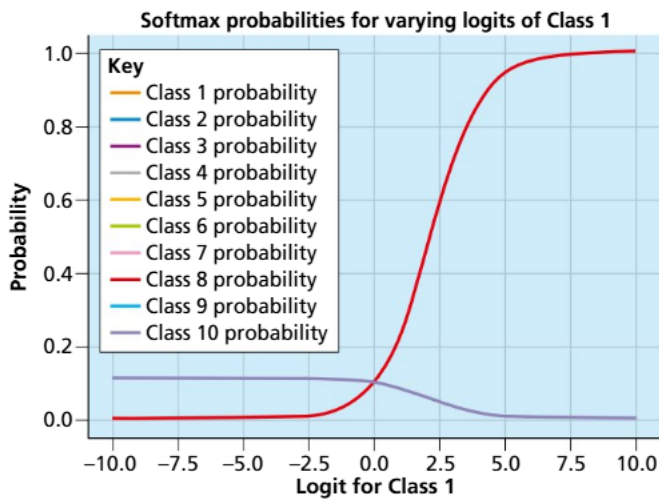
On first impression, it may appear that ReLU is linear; however, it is more accurate to say it is two different lines coming together in the one function, one on the positive side, and another on the negative. The simple act of the zeroing of negative values significantly changes the behaviour within a network as it means that only positive neurons will be activated and any negative-value neurons will be deactivated. This small change makes a big difference when attempting to do classification problems.

Sigmoid

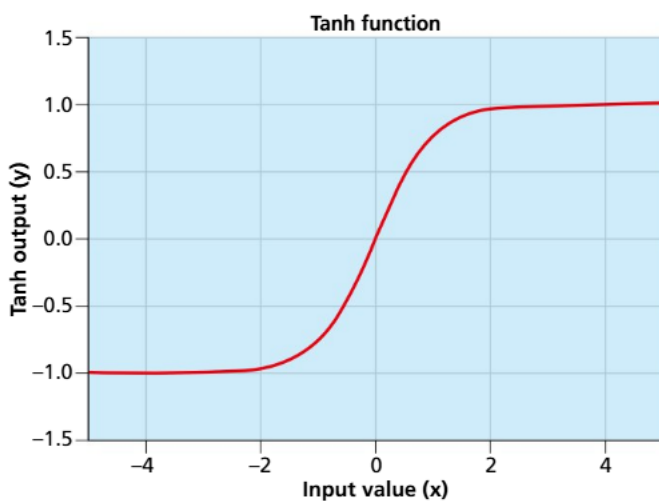
Sigmoid is commonly used in the output layer for binary classification problems since it maps to a distribution between 0 and 1, which is generally what is desired at the output layer. It is used in scenarios like email spam detection (spam or not spam) and medical diagnosis (sick or healthy). It is not usually used in hidden layers in deep networks due to their vanishing gradients. Observe that once the input value is less than -4 or greater than $+4$, the gradient becomes so insignificant it might as well be zero.

The equation for Sigmoid is:

$$f(x) = \frac{1}{1 + e^{-x}}$$



■ The Softmax function



■ The tanh function

Softmax

Softmax produces an output similar to Sigmoid in that both produce values in the range (0,1). Softmax produces a probability distribution for N different outcomes, where N is the number of categories for classification and the probabilities sum to 1. This makes it suitable for distribution across multiple classes, so it is commonly used for the output layer of a multiclass classification problem. (In the chart, the lines for Classes 2 to 10 are aligned and stacked one on the other, which is why it appears as if only two lines are plotted.)

Tanh

Tanh is similar to Sigmoid, but the output values range between -1.0 and 1.0. It is useful when your data is normalized around 0 but, like Sigmoid, it also has vanishing gradients that can be problematic. It is more common to see tanh used for hidden layers than Sigmoid, given its mean distribution is centred on 0. This centring around 0 makes learning for the next layer easier for classification.

There are a couple of mathematically equivalent ways of producing the tanh function:

$$f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$$

or

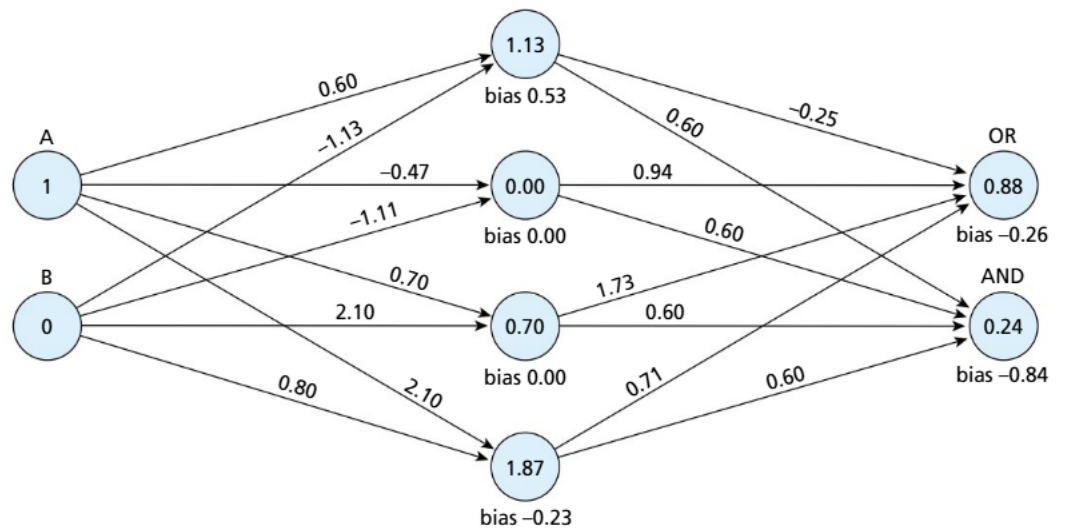
$$f(x) = \frac{2}{(1 + e^{-2x})} - 1$$

Generating a prediction

Using an ANN to generate a result is a matter of performing all the calculations on all the perceptrons in one layer, and then feeding forward those results to the next layer. The process continues until the output layer is reached and the process terminates.

While overengineered for the scenario, imagine using a neural network to determine the result of OR and AND logic gates. Consider the following network with two input neurons, four neurons in one hidden layer and two output neurons.

After training (discussed in the next section), the network consists of the following weights and biases. The ReLU activation function is used on the hidden layer and, since the network is seeking to perform a classification task, Sigmoid is used on the output layer.



■ Example values for a logic gate ANN

For those unfamiliar with logic gates, see Section A1.2.3. The network should produce the following results, if behaving correctly:

A	B	OR	AND
[0, 0]	->	[0, 0]	
[0, 1]	->	[1, 0]	
[1, 0]	->	[1, 0]	
[1, 1]	->	[1, 1]	

Performing the calculations for an input of [1, 0], the following occurs:

$$\begin{aligned} \text{Hidden1} &= \text{ReLU}(1.00 * 0.60 + 0.00 * -1.13 + 0.53) = \text{ReLU}(1.13) = 1.13 \\ \text{Hidden2} &= \text{ReLU}(1.00 * -0.47 + 0.00 * -1.11 + 0.00) = \text{ReLU}(-0.47) = 0.00 \\ \text{Hidden3} &= \text{ReLU}(1.00 * 0.70 + 0.00 * 2.10 + 0.00) = \text{ReLU}(0.70) = 0.70 \\ \text{Hidden4} &= \text{ReLU}(1.00 * 2.10 + 0.00 * 0.80 - 0.23) = \text{ReLU}(1.87) = 1.87 \end{aligned}$$

Now, use these hidden layer values to generate the output values.

$$\begin{aligned} \text{Out1} &= \text{Sigmoid}(1.13 * -0.25 + 0.00 * 0.94 + 0.70 * 1.73 + 1.87 * 0.71 - 0.26) \\ &= \text{Sigmoid}(2.00) \\ &= 0.88 \end{aligned}$$

$$\begin{aligned} \text{Out2} &= \text{Sigmoid}(1.13 * -1.48 + 0.00 * -0.69 + 0.70 * -0.13 + 1.87 * 0.77 - 0.84) \\ &= \text{Sigmoid}(-1.16) \\ &= 0.24 \end{aligned}$$

Since our classification problem is seeking a 0 (“false” or “no”) or 1 (“true” or “yes”) answer, when 0.88 and 0.24 are rounded, the network has indeed correctly determined that an input of [1,0] into an OR gate results in a 1, and [1,0] into an AND gate results in a 0.

● See also

See 3Blue1Brown’s YouTube video “But what is a neural network?”

◆ **Backpropagation:** backpropagation of errors is the most commonly used technique for training artificial neural networks. The gradient of the loss function is calculated, and used to update parameters such as weights, in the opposite direction of the gradient to reduce the overall error.

■ Training

While calculating an output result or prediction from a neural network should be a conceptually straightforward mathematical process, the training process is more complex and is far beyond the scope of your course.

The process used is known as **backpropagation**. As a high-level overview, here is what is occurring:

- We calculate the error in the output values received from the network when compared to the target output values in the training data. A loss function is used for this, such as “mean-squared-error” for regression, or “cross-entropy loss” for classification tasks.
- We calculate how much each parameter (the weights and biases) in the network contributed to the error. This is done by using the gradient (i.e. the derivative or slope) of the loss function for each parameter.
- An optimization algorithm such as “gradient-descent” is used to calculate adjustments to the parameters. By knowing the gradient of the error, the parameters can be adjusted in the opposite direction (gradient descent) to reduce the loss (see the videos referenced in the “See also” box).
- Before applying the adjustment to the weights and biases, we multiply them by the learning rate hyperparameter. This is to ensure we don’t overcorrect and solely design the network around any one particular value in the training data set.
- Once this process has completed for one layer (such as using the output layer to calculate adjustments to the last hidden layer), we repeat the process on the layers before it. This process of moving backwards from the output layer, working through each hidden layer, until finally reaching the input layer, is where the term “backpropagation” comes from.
- We repeat the entire process a certain number of iterations or until the loss stops decreasing significantly. Each pass over the data set is known as an “epoch”.

● See also

See 3Blue1Brown’s YouTube videos “Backpropagation, step-by-step | DL3” and “Gradient descent, how neural networks learn”.

■ Example 1: Logic gates

This is the Python code used for the OR and AND logic gates example in the previous walkthroughs.

Python

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
# Inputs: [A, B]
x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
# Outputs: [OR, AND]
y = np.array([[0, 0], [1, 0], [1, 0], [1, 1]], dtype=float)
```

```
# Define and compile the model
model = Sequential([
    # Hidden layer with 4 neurons, using ReLU
    Dense(4, input_dim=2, activation="relu"),
    # Output layer with 2 neurons, using Sigmoid
    Dense(2, activation="sigmoid")
])
model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
model.fit(x, y, epochs=1000, verbose=1)
# Making predictions
predictions = model.predict(x)
print("Predicted outputs:\n", predictions)
# Evaluate the model
loss, accuracy = model.evaluate(x, y)
print("Accuracy: {:.2f}".format(accuracy))
# Print weights and biases for our curiosity
for layer_number, layer in enumerate(model.layers):
    weights, biases = layer.get_weights()
    print(f"Layer {layer_number+1}")
    print("Weights:\n", weights)
    print("Biases:\n", biases)
    print("\n")
```

Common mistake

If you are installing TensorFlow on a computer without a GPU, ensure you install a CPU-only version otherwise you will receive errors that pip is unable to find a version that satisfies the requirements.

That is, from your terminal, run the following:

```
pip install tensorflow-cpu
```

For more detailed instructions, refer to the TensorFlow installation guide at www.tensorflow.org/install/pip

■ Example 2: ANN for regression

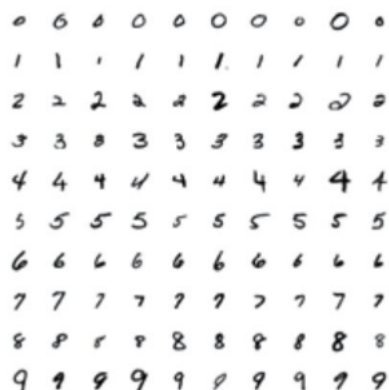
A commonly used example for introducing regression problems with an ANN is the California housing data set. It contains information about various homes in California in the 1990s, including such features as house age, average number of rooms, average number of residents, and latitude and longitude, and is used to predict house prices.

More information on the data set can be found by searching online for Keras California Housing price regression data set (https://keras.io/api/datasets/california_housing).

Python

```
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
import numpy as np
```

```
# Load data set
(x, y), (x_test, y_test) = tf.keras.datasets.california_housing.load_data(
    version="large", path="california_housing.npz", test_split=0.2, seed=113
)
# Split the data into training and validation sets
# x_train is the training data, y_train is the training labels
# x_val is the validation data, y_val is the validation labels
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=0.2,
    random_state=0)
# Define the network, compile, and train it
model = models.Sequential([
    layers.Dense(64, activation="relu", input_shape=(x_train.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dense(1)
])
model.compile(optimizer="adam",
    loss="mse",
    metrics=["mae"])
history = model.fit(x_train, y_train,
    epochs=100,
    validation_data=(x_val, y_val))
# Run the unseen test data through the network to determine success
test_loss, test_mae = model.evaluate(x_test, y_test)
print(f"Test data - mean-absolute-error: {test_mae}")
```



■ Examples of handwritten digits in the MNIST data set

■ Example 3: ANN for classification

A very common Hello World-style classification problem for ANNs is the MNIST number recognition data set. It comprises 60,000 28x28 grayscale images of the ten digits, along with a test set of 10,000 images.

Python

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np
# Load data set
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```



```

# Convert grayscale pixels into floats with range [0...1]
x_train = x_train/255.0
x_test = x_test/255.0
# Convert the labels into "1 hot encoding" category arrays
# e.g. label of 3 becomes [0,0,0,1,0,0,0,0,0,0]
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
# Define the network, compile, and train it
model = models.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(64, activation="relu", input_shape=(x_train.shape[1],)),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="sigmoid")
])
model.compile(optimizer="adam",
              loss="binary_crossentropy",
              metrics=["accuracy"])
history = model.fit(x_train, y_train, epochs=5)
# Run the unseen test data through the network to determine success
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Accuracy: {accuracy}")
model.save("mnist-example.keras")

```

One step that may not be intuitively obvious is how to test this with your own data. Suppose you have a 28x28 grayscale PNG file you'd like to test on the ANN. The following is example code to do this task. *(By the way, ensure your image is white text on black background, as that is how the model has been trained.)*

Python

```

import numpy as np
from PIL import Image      # pip install Pillow
import tensorflow as tf
model = tf.keras.models.load_model("mnist-example.keras")
# Load the image file
image = Image.open("your image file.png")
image = image.convert("L") # Grayscale to match the training data
image = image.resize((28, 28)) # Resize to match the training data
# Convert image to numpy array
image_array = np.array(image)
image_array = image_array/255.0 # Normalize to 0..1 scale
# Reshape the array for the model (Add a batch dimension at the beginning)
image_array = image_array.reshape(1, 28, 28)
# Send to the trained ANN
predictions = model.predict(image_array)
predicted_class = np.argmax(predictions, axis=1)
print("Predicted class:", predicted_class)

```

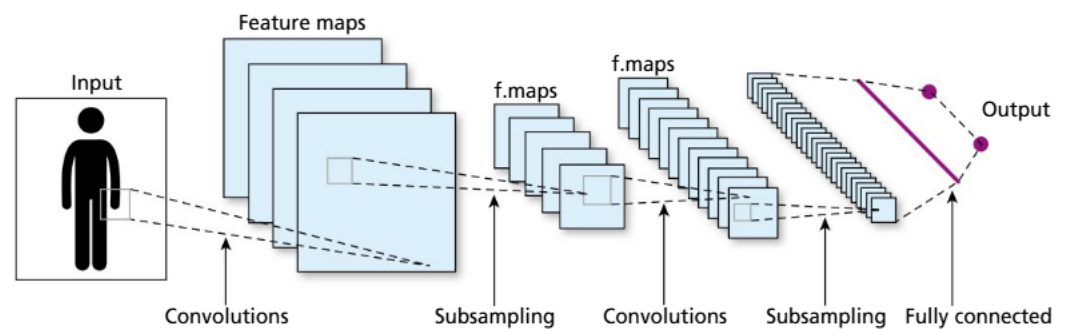
A4.3.9 Convolutional neural networks

◆ **Convolution:** a mathematical operation that combines two functions to produce a third function. In the context of a convolutional neural network being used for image processing, convolution applies filtering functions to the pixels in an input image to compute distinctive features from the data.

A **convolutional neural network** (CNN) extends on the architecture of ANNs by using additional layers of calculations prior to processing the data through a fully connected artificial neural network.

CNNs are ideally suited to processing image data but are also valuable for applications such as video analysis, natural language processing (NLP), audio and speech processing, and recommendation systems.

To explore what makes a CNN different from an ANN, consider the following diagram of a typical convolutional neural network:



■ Structure of a convolutional neural network

■ Input layer

The input layer comprises the raw pixel data of the image being processed. The number of input nodes would be based upon $\text{image width} \times \text{image height} \times \text{colour depth}$.

It is important to consider that, when dealing with input data such as images, the input data can get very large with relative ease. A small image of only 100×100 pixels at full colour resolution (that is, 1 byte each for red, green and blue) would comprise 300,000 values. This is too much to feed directly into an ANN, and so preprocessing through convolution and pooling is used to reduce this to something more manageable.

■ Convolutional layer

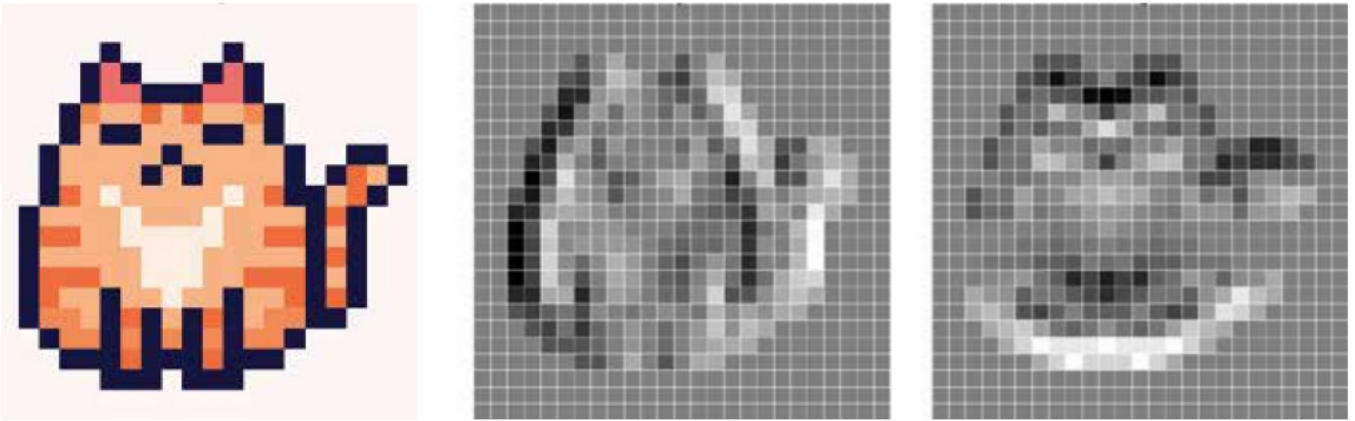
The convolutional layers serve as feature extractors that are looking for patterns in the input image.

As the network trains, it develops filters (also known as “kernels”) that learn to detect patterns that are important for the individual network at hand. Generally, these patterns are as simple as edge detection or various textures, but they can be used to detect more complex shapes within the image. Other common patterns that may emerge are sharpening filters and blur filters.

Edge detection will typically look for vertical or horizontal edges. For instance, the matrix to find vertical edges may look like this:

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Applying an edge detection filter to an image is shown in the following diagram. The first image is the original; the second is the output using vertical edge detection with the matrix above; the third is horizontal edge detection after rotating the matrix clockwise.



■ Kitten bitmap with its horizontal edges and vertical edges detected

A sharpening filter will emphasize the difference in contrast between adjacent pixel values. This helps make the image look more distinct to later stages of the network. The matrix for a sharpening filter may look like this:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

A blur filter will help reduce noise and detail from an image, so the network doesn't focus on minor variations of detail within the image that do not carry significance in meaning. For instance, a gaussian blur may be applied through the following matrix:

$$\begin{bmatrix} 1/16 & 1/8 & 1/16 \\ 1/8 & 1/4 & 1/8 \\ 1/16 & 1/8 & 1/16 \end{bmatrix}$$

The strength of CNNs lies in their ability to learn the most appropriate filters for a given task through backpropagation during the training process. The initial values for these kernels can be set randomly, or by using some heuristic, and then be updated to better suit the specific features of the training data.

The act of iterating over groups of pixels with a filter is known as “sliding” or “striding”.

● See also

See 3Blue1Brown's YouTube video “But what is a convolution?”

■ Activation function

Convolutional operations are linear transformations. Mathematically, they are the dot product between the filter (kernel) values and the pixel values in the image. If you stack multiple convolutional layers, the entire network can still be described as a single linear transformation. No matter how many layers you have, they can still be collapsed into a single layer that performs one linear transformation, because the composition of two linear functions is still a linear function (the function for any one pixel can be reduced to the sum of input values multiplied by various fixed coefficients).

This poses a problem for our network, as a linear system can only perform linear classification. That is, it can only separate data using a straight line (or hyperplane in higher dimensions). While this has its uses, the tasks we generally require of CNNs are too complex for a linear approach to separation.

Therefore, after convolution, the data is run through an activation function to introduce non-linearity to the data.

■ Pooling layer

The next stage of the process is to pass the data through some down-sampling layers, also known as “pooling layers”. The pooling layers serve to reduce the dimensions of the image data. This serves a couple of important purposes. Firstly, it will reduce the number of parameters that need to be input into a deeply connected network, significantly reducing the computational workload of the network. Secondly, it also assists with network learning, as it will further help negate minor changes in individual pixels (image noise) that are unlikely to be relevant to image classification. This helps to reduce the risk of overfitting.

There are two commonly used methods of pooling:

- **Max pooling**, which takes the maximum value from a set of values.
- **Average pooling**, which takes the average value from the set of values.

■ Fully connected layer

After pooling, the data is then fed into a fully connected artificial neural network (ANN), which has been previously discussed.

The purpose of the ANN at this point is to take the high-level features learned by the convolutional and pooling layers and use them to perform the final classification or regression task.

Before entering the first fully connected layer, the feature maps will typically be flattened into a one-dimensional vector of values, which will align with the number of input nodes of the ANN.

■ Output layer

The output layer from the CNN is the output that comes from the fully connected layers of the ANN.

● Top tip!

Don't neglect transfer learning to leverage pre-trained models. This is especially useful if you have limited training data available.

- Explore some of the pre-trained models available through TensorFlow or PyTorch, such as VGG, ResNet and Inception.
- Consider the similarity between the data the model was originally trained on, and your target data.
- Decide whether you will be using transfer learning to assist with feature extraction or fine-tuning. Feature extraction will use the previous network to extract meaningful features from new samples. In this case, freeze the convolutional base and only train a new classifier layer. Fine-tuning doesn't freeze the convolutional base. After adding your new output layer, it will fine-tune the weights of the pre-trained model by continuing the training process, allowing it to learn new task-specific features.
- Remember to preprocess data in the same way the original model was trained, and to use data augmentation techniques (like rotation, scaling, cropping and flipping) to artificially expand the training data set.

■ Example: CIFAR-10

The CIFAR-10 data set contains 50,000 images of 32×32 pixels in RGB colour, plus another 10,000 test images. The images are labelled over ten categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

The additional complexity of shapes, slightly enlarged size and the inclusion of three colour channels rather than just grayscale make CIFAR a good platform for experimenting with a CNN.

More information about the data set can be found by searching online for Keras CIFAR10 small images classification data set (<https://keras.io/api/datasets/cifar10>).

● TOK

Is it acceptable to benefit from knowledge derived from unethical sources?

CIFAR-10 is a data set that was compiled by Alex Krizhevsky in 2009. It was created as a subset of a larger data set of 80 million images known as Tiny Images.

In June 2020, the decision was made to withdraw the Tiny Images data set and request others to stop using it, due to “biases, offensive and prejudicial images, and derogatory terminology” within the data set.

CIFAR-10 remains available and is commonly used by many academic institutions. Students use it to learn how to train neural networks for computer vision tasks.

Given its origins from the ethically compromised Tiny Images data set, should CIFAR-10 still be used in scientific research and technological development?

● TOK

Is it ethical to use data scraping for creating data sets without the consent of the content owners?

Web scraping is a prevalent tool used to gather vast amounts of data from across the internet. Typically, this occurs without the explicit consent of the owners or creators of the data.

Web scraping is fundamental to the data sets used for many machine learning models, including computer vision and large language models.

Courts and governments are grappling with the complex ethical issues around this practice, and no clear resolution is in sight. There are vast economic and commercial interests on both sides of the debate.

Some questions include:

- Who owns the information available on the internet, especially on forums such as Reddit, or collaborative efforts such as Wikipedia?
- Is it ethical to use this data for academic purposes? What about for commercial purposes?
- Is it an invasion of privacy to use photos and videos uploaded to social media to form data sets for machine learning purposes?
- Is it too late? Is it time to focus on harm mitigation? Are there ways to share profits, such as through royalty payments?

PYTHON

```

from tensorflow.keras.datasets import cifar10
from tensorflow.keras import layers, models
import numpy as np
# Load the CIFAR-10 data set
(train_images, train_labels), (test_images, test_labels) = cifar10.load_data()
# Normalize the data to 0-1 ranges
train_images = train_images/255.0
test_images = test_images/255.0
# Define the CNN model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation="relu", input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
# Compile and train the model
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
history = model.fit(train_images, train_labels, epochs=10, validation_split=0.1)
# Evaluate the model
test_loss, test_acc = model.evaluate(test_images, test_labels)
print("Test accuracy:", test_acc)

```

 Top tip!

Start simple! Begin with simple models to establish a baseline, and gradually move to more complex algorithms. Appreciate the power of simple models; sometimes they are all you need.

A4.3.10 Model selection

You have looked at a lot of machine learning algorithms in this chapter. When the time comes to use machine learning to solve your own problems, how do you decide which model to use? Here are some criteria to assist in your decision-making process:

- **Classification or regression?** Does the problem require predicting a continuous output (regression) or categorizing data into predefined classes (classification)?
- **Linear or non-linear relationship?** Linear regression is quick and simple to implement, but will not work with complex non-linear data, in which case a neural network may be required.
- **Low or high dimensionality?** How many features do you need your algorithm to process?
- **Volume of data?** Deep learning requires a large amount of data to work accurately and to avoid overfitting. Decision trees or k-nearest neighbours may be better suited if the data set is small.
- **Feature independence?** If features (variables) interact with each other (such as co-dependency), the complex interplay may be better captured by a decision tree or neural network.
- **Accuracy?** If highly accurate predictions are required, then more complex models may be the better option, but this comes at the cost of requiring more data and computational power.

ACTIVITY

Thinking skills: As a class, brainstorm a set of case-study scenarios where machine learning may be beneficial. Debate and discuss what the appropriate machine learning algorithm would be (for example linear regression, clustering, association rules) based on the problem statement and data characteristics.

- **Training time?** Linear regression and shallow decision trees can be trained very quickly, relative to deep neural networks.
- **Transparency?** Sometimes the “magic-happens-here” approach of neural networks may be intimidating and undesired by the client. Some domains, such as healthcare or finance, may require models that can be user-interpreted, in which case linear regression or decision trees may be best.
- **Resources available?** Deep neural networks require significant GPU computational power to train. If all you have is a consumer-grade laptop, a simpler approach may be required.

Key information

Remember that, as far as the syllabus is concerned, Machine Learning is a theory unit rather than a programming one. The following exercises are optional suggestions for students who wish to explore machine learning programming for themselves.

PROGRAMMING EXERCISES

1 Height and weight (linear regression)

Given the height of a person, can you predict their weight?

Data set @ www.kaggle.com/datasets/galserge/weight-and-height-from-nhanes

2 Vide ogame sales with ratings (linear regression)

Given the ratings assigned by critics reviewing a new Video game, can you predict how many millions of units a Video game will sell?

Data set @ www.kaggle.com/datasets/rush4ratio/video-game-sales-with-ratings

3 Societal impact on education (linear regression)

How much is a student’s educational outcome influenced by the strength of the economy and health of the society in which they reside?

Data set @ www.kaggle.com/datasets/walassetomaz/pisa-results-2000-2022-economics-and-education

4 Zoo animal classification (k-nearest neighbours)

Classify animals into categories, such as mammal, bird or reptile, based on attributes such as weight, height and type of habitat.

Data set @ www.kaggle.com/datasets/uciml/zoo-animal-classification

5 Mall customer segmentation (unsupervised k-means clustering)

Use k-means clustering to identify distinct customer groups, such as high-income–high-spending vs low-income–high frequency customers.

Data set @ www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

6 Social network analysis (unsupervised spectral clustering)

Discover socially connected communities within the Zachary karate club data set. One challenge often asked with this data set is to find the two groups of people into which the karate club split after an argument between two of the teachers.

Search “Zachary karate club” to find this data set.

7 (Unsupervised association rule learning)

Analyse the grocery-store data set to discover common product combinations purchased together.

Data set @ <https://archive.ics.uci.edu/dataset/611>

8 (Reinforcement learning)

The previously mentioned Gymnasium has a number of pre-built environments for you to experiment with.

Download @ <https://gymnasium.farama.org>

9 Optimal stock portfolio (genetic algorithm)

Use a genetic algorithm to determine what would be the optimal mix of stocks to hold over the duration of a data set to maximize return while minimizing risk. The fitness function could be based on the Sharpe ratio, a measure of return adjusted for risk.

Download historical price data for a set of assets (for example stocks, bonds, ETFs) and calculate returns for each asset to use in the optimization.

Data set @ www.kaggle.com/datasets/jacksoncrow/stock-market-dataset

Data set 2 @ www.nasdaq.com/market-activity/quotes/historical

10 Stock-price prediction (artificial neural network)

Can you create an AI to accurately predict the performance of stock prices? (If you can, don't forget to express your appreciation benevolently to the textbook authors 😊)

Data set @ www.kaggle.com/datasets/jacksoncrow/stock-market-dataset

Data set 2 @ www.nasdaq.com/market-activity/quotes/historical

11 Cats and dogs (convolutional neural network)

Can you tell the difference between a cat and a dog?

Data set @ www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset

12 Traffic-sign recognition (convolutional neural network)

Accurately detecting road signs is a core challenge for the development of self-driving cars. The traffic-sign recognition data set contains over 50,000 images across 40 classes of road sign. Should we let you develop the AI for the next breed of self-driving cars?

Data set @ www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign

13 Movie-reviews sentiment analysis (choose between ANN and CNN)

This exercise will introduce you to natural language processing. Specifically, you will use sentiment analysis to predict positive and negative reviews based on movie reviews on IMDb.

Data set @ www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

Be aware that this exercise will involve learning several additional important concepts to implement. This is because the data set is text, but machine learning models require numeric data to function, so significant preparation and preprocessing of your data is required.

The following tips will guide you:

- ☐ Convert text to lowercase and remove non-alphabetic characters.
- ☐ Tokenize the words, which is the process of splitting the text into individual words or word parts. For instance, the string "hello world" would be tokenized into ["hello", "world"]. Refer to www.nltk.org/api/nltk.tokenize.html
- ☐ Remove stop words, which are words that generally don't convey meaning. Examples include "a", "the" and "and". Refer to <https://pythonspot.com/nltk-stop-words>
- ☐ Use a vectorizer such as CountVectorizer or TfidfVectorizer to transform your text into numeric vectors. Refer to https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction
- ☐ Convert your vectors into NumPy arrays.
- ☐ Create TensorFlow data sets from the NumPy arrays. Refer to www.tensorflow.org/api_docs/python/tf/data/Dataset

Now you can build a neural network model and train it.

REVIEW QUESTIONS

- 1 An e-commerce company uses linear regression to predict customer spending based on their past purchasing behaviour.
 - a State the assumption about the relationship between the dependent and independent variables in linear regression.
 - b Describe how outliers could affect the performance of the linear regression model in this scenario.
 - c Describe one method to evaluate the accuracy of this linear regression model.
- 2 A real-estate company uses linear regression to estimate property prices based on features like area, age and number of rooms. One of the technical staff expressed concern that multicollinearity might be a problem with the model. Multicollinearity is when two or more independent variables have a high correlation with one another in a regression.
 - a Explain why multicollinearity might be a problem in this linear regression model.
 - b Outline a method to handle multicollinearity if it is found in the data set.
- 3 A college uses linear regression to predict student success based on high-school GPA, standardized test scores and college entrance essays.
 - a Outline one reason why it is important to assume linearity in this regression model.
 - b Suggest a technique to assess the model's predictive accuracy and explain its importance.
- 4 A medical research institution develops a decision tree model to classify patients into risk categories for heart disease based on lifestyle and genetic data.
 - a Describe one advantage of using decision trees for this type of classification problem.
 - b Describe one disadvantage of using decision trees for this type of classification problem.
 - c
 - i Identify one critical parameter in decision trees that could impact the model's performance.
 - ii Outline its role.
- 5 An online retailer uses k-nearest neighbours (KNN) to classify customer reviews as positive, neutral or negative.
 - a Outline how the choice of k affects the classification accuracy in KNN.
 - b Describe one method to determine the optimal k value for this application.
 - c Describe how the scales used by features influence the performance of the KNN algorithm.
- 6 A high school wants to classify students into different learning groups based on their learning styles and previous academic performance.
 - a Outline two reasons to select decision trees over KNN for this problem.
 - b Outline two reasons to select KNN over decision trees for this problem.
 - c The school decided to use a decision tree. Describe one strategy to prevent overfitting in the decision tree model.
- 7 A maintenance system uses supervised learning to forecast equipment failures in an industrial plant.
 - a Define "precision" and "recall" in the context of this predictive system.
 - b Explain why the F1 score is a better measure than accuracy in scenarios where false negatives have higher costs.
 - c Describe how a confusion matrix can be used to visually illustrate the success of the model.

- 8 A health diagnostic application uses supervised learning to classify patient results as “normal” or “abnormal”.
 - a Outline the importance of a high recall rate in this medical classification task.
 - b Describe how an imbalanced data set might affect the performance metrics like precision and recall.
 - c
 - i Identify one method to adjust the classification threshold.
 - ii Describe its impact on the F1 score.
- 9 An online retailer uses k-means clustering to segment customers based on purchasing patterns.
 - a Outline the objective of the k-means clustering algorithm.
 - b Describe one challenge when using k-means clustering for customer segmentation.
 - c Describe how the choice of k affects the outcomes of the k-means algorithm.
- 10 A telecommunications company uses spectral clustering to segment customers based on usage patterns.
 - a Describe the difference between k-means and spectral clustering in handling non-spherical data clusters.
 - b Describe one challenge in using spectral clustering for large data sets.
 - c Describe how the results of spectral clustering could be used to improve customer satisfaction.
- 11 A social-media company uses clustering to identify social groups on its network system.
 - a Identify which clustering algorithm would allow identification of social groups in this network.
 - b Describe one potential challenge in clustering users based on such diverse data.
 - c Describe how the choice of the number of clusters can affect the results.
- 12 Urban planners in a large city are using data collected from traffic sensors at various intersections and highways to identify clusters of intersections and road segments that exhibit similar traffic patterns.
 - a Describe a suitable algorithm that the urban planners could use to group sensor data into clusters based on their traffic characteristics. Explain why this algorithm is appropriate for handling data with varying densities and noise.
 - b Describe how understanding these traffic clusters could benefit the city’s traffic management and infrastructure planning.
- 13 An e-commerce platform analyses user purchasing data to discover frequent buying patterns.
 - a Define “lift” in the context of association rule mining and its importance.
 - b Describe how minimum support and confidence levels affect the rules generated in this scenario.
 - c Describe the potential impact of these buying patterns on targeted marketing strategies.
- 14 A library analyses borrowing patterns to find associations between different genres of books borrowed together.
 - a Define “confidence” and “support” in association rule mining for this library data.
 - b Describe how the library can discover these patterns.
 - c Describe one potential limitation of association rule mining in predicting book-borrowing patterns.
- 15 An engineering firm uses genetic algorithms to optimize the design of a new aerodynamic vehicle model.
 - a Outline the role of crossover in genetic algorithms.
 - b Outline how mutation affects the evolution process in genetic algorithms.
 - c Outline one advantage of using genetic algorithms in complex optimization problems such as vehicle design.

- 16 A school runs an elective block on the timetable where students can select from a number of creative and optional courses. Students are asked to indicate their preferred courses but are not guaranteed to receive their first preference. The school uses a genetic algorithm to maximize the number of students receiving their first or second preference.
 - a Outline the function of selection in genetic algorithms.
 - b Describe the concept of “fitness function” in genetic algorithms, and how it might be applied in this scenario.
 - c Describe how population size influences the outcome of a genetic algorithm.
 - d Outline two benefits of using genetic algorithms to design a timetable schedule.
 - e Outline two drawbacks of using genetic algorithms to design a timetable schedule.
- 17 A financial institution employs an artificial neural network to predict loan default risk based on customer profiles.
 - a Identify one type of layer often used in neural networks and its purpose.
 - b Describe why overfitting might be a concern in neural networks.
 - c Describe how a neural network can be trained to minimize prediction error in this financial context.
- 18 An energy company uses a neural network to forecast electricity demand based on weather conditions and historical usage.
 - a
 - i Identify whether the neural network in this scenario would be regression or classification based.
 - ii Outline the significance of that on its design.
 - b Outline one type of activation function used in neural networks and its purpose.
 - c Outline which activation function would most likely be suitable for the output layer in this scenario, and why.
 - d Describe why deep neural networks might be more effective than shallow networks for this forecasting task.
 - e Define “backpropagation” and outline its role in learning within a neural network.
 - f Outline two challenges associated with training deep neural networks.
- 19 A tech company experiments with several machine learning models to predict user engagement on a new app.
 - a Define “model selection” in the context of machine learning.
 - b Identify two metrics that could be used to select the best model for predicting user engagement.
 - c
 - i Outline the concept of cross-validation.
 - ii Describe one reason why it is important in model selection.
- 20 A sports analytics company tests multiple models to predict the outcome of basketball games.
 - a Describe the concept of “overfitting” in the context of model selection.
 - b Identify three factors that should be considered in model selection.